IT5412
ADVANCED DATA STRUCTURES LABORATORY

A PRACTICAL RECORD

Submitted By

Yuvaraj K
2022506079
B. Tech Information Technology (4/8)

DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY
ANNA UNIVERSITY, CHENNAI – 600 044

January – May 2024

BONAFIDE CERTIFICATE

Name	**:** Yuvaraj K
Register No.	**:** 2022506079
Subject	**:** IT5412 Advanced Data Structures Laboratory
Department	**:** Information Technology

Certified to the bonafide record of practical work done by Mr. Yuvaraj K in the IT5412 Advanced Data Structures Laboratory during the period January 2024 to May 2024.

Date:

Submitted for the Practical Examination held on: 07/05/2024

Dr. J. Dhalia Sweetlin
STAFF-IN-CHARGE

List of Experiments

Exp. No	Date	Name of the Experiment	Page	Signature
1	06/02/24	IMPLEMENTATION OF AN APPLICATION USING CLASSES, OBJECTS, CONSTRUCTORS, DESTRUCTORS, AND STRING HANDLING	1	
2	13/02/24	IMPLEMENTATION OF PROGRAMS USING FUNCTION OVERLOADING AND OPERATOR OVERLOADING	23	
3	20/02/24	IMPLEMENTATION OF AN APPLICATION USING INHERITANCE, VIRTUAL FUNCTIONS, AND ABSTRACT CLASSES	57	
4	27/02/24	IMPLEMENTATION OF PROGRAMS USING FUNCTION TEMPLATES AND CLASS TEMPLATES	145	
5	05/03/24	IMPLEMENTATION OF A SUPERMARKET MANAGEMENT SYSTEM USING OBJECT ORIENTED PROGRAMMING CONCEPTS	153	
6	12/03/24	IMPLEMENTATION OF STACK, QUEUE, AND LIST DATA STRUCTURES USING STL CONCEPTS	225	

7	19/03/24	IMPLEMENTATION OF AVL TREE USING TEMPLATES	247	
8	26/03/24	IMPLEMENTATION OF SPLAY TREE USING TEMPLATES	269	
9	02/04/24	IMPLEMENTATION OF A HEAP TREE USING TEMPLATES	289	
10	09/04/24	IMPLEMENTATION OF GRAPH TRAVERSAL ALGORITHMS: BREADTH-FIRST SEARCH, DEPTH-FIRST SEARCH	301	
11	16/04/24	IMPLEMENTATION OF SHORTEST PATH ALGORITHM: DIJKSTRA’S ALGORITHM	319	
12	23/04/24	IMPLEMENTATION OF MINIMUM SPANNING TREE: PRIM’S ALGORITHM	329	

Exp. No: 1
Date: 13/02/2024

IMPLEMENTATION OF AN APPLICATION USING CLASSES, OBJECTS, CONSTRUCTORS, DESTRUCTORS, AND STRING HANDLING

AIM

To implement a supermarket management system application using classes, objects, constructors, destructors, and string handling.

ALGORITHM

All functionalities are from the employee perspective:

- Adding a new customer
- Adding new products/items
- Updating product stock
- Creating a new bill
- Viewing customer, product, and bill details

```
class Customer() {
        name, phone no;
        getCustomerDetails();
        updateDetails();
        displayDetails();
}
class Product {
        id, name, price, quantity;
        getProductDetails();
        viewProductDetails();
}
class Bill {
        bill_id, products, quantities;
        createBill();
        displayBill();
}
```

SOURCE CODE

supermarket_class.h

```
#include <iostream>
#include <string>
using namespace std;
class Customer {
	public:
		string phone_no;
		string name;
		string temp;
		int isValid(string pno) {
			if (pno.length()!=10) {
				cout<<"Invalid Phone Number."<<endl;
				return 0;
			}
			return 1;
		}
		int getCustomerDetails() {
			getline(cin, temp);
			cout<<"\nPhone Number: ";
			getline(cin, phone_no);
			if (isValid(phone_no)) {
				cout<<"Name: ";
				getline(cin, name);
				return 1;
			}
			return 0;
		}
		int updatePhoneNo (string newno) {
			if (isValid(newno)) {
				phone_no=newno;
				return 1;
			}
			return 0;
		}
		void updateName (string newname) {
			name=newname;
		}
```

```
		void displayCustomerDetails() {
			cout<<"\nPhone No: "<<phone_no<<endl;
			cout<<"Customer Name: "<<name<<endl;
		}
};
class Product {
	public:
		int product_id, quantity;
		float price;
		string product_name;
		Product() {}
		Product(int id) {
			product_id=id;
			product_name="Unnamed";
			price=500;
			quantity=10;
		}
		Product (int id, float p, int q) {
			product_id=id;
			product_name="Unnamed";
			price=p;
			quantity=q;
		}
		Product (int id, string name, int q) {
			product_id=id;
			product_name=name;
			cout<<product_name;
			price=500;
			quantity=q;
		}
		Product (int id, float p, string name, int q) {
			product_id=id;
			product_name=name;
			price=p;
			quantity=q;
		}
		void updateStock (int q) {
			quantity+=q;
			cout<<"\nQuantity of "<<product_name<<" successfully
updated."<<endl;
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_01$ g++ supermarket.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_01$ ./a.out

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 1

Phone Number: 7550099095
Name: Yuvaraj K
Customer Added.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 4

Phone Number: 7550099095

Phone No: 7550099095
Customer Name: Yuvaraj K
```

```
		cout<<"New Quantity = "<<quantity<<endl;
	}
	void displayProductDetails() {
		cout<<"\nProduct Name: "<<product_name<<endl;
		cout<<"Price: "<<price<<endl;
		cout<<"Available Quantity: "<<quantity<<endl;
	}
};
class Bill {
	public:
		int bill_id, no_of_products;
		Customer customer;
		Product * products=new Product[3];
		float total;
		void createBill (int id) {
			bill_id=id;
			no_of_products=0;
		}
		void getCustomer(Customer c) {
			customer=c;
		}
		int checkProduct(Product p) {
			for (int i=0; i<no_of_products; i++)
				if (products[i].product_id==p.product_id)
					return i;
			return -1;
		}
		void addProduct(Product p) {
			int pos=checkProduct(p);
			if (pos>=0) {
				products[pos].quantity+=p.quantity;
				cout<<p.product_name<<" already exists. "<<p.quantity<<"
units of "<<p.product_name<<" has been added."<<endl;
			} else {
				products[no_of_products]=p;
				cout<<p.quantity<<" units of "<<p.product_name<<" has
been added."<<endl;
				no_of_products++;
			}
}
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 2

Enter Old Phone Number: 7550099095
Enter New Phone Number: 8939314400
Update Successful.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 3

Phone Number: 8939314400
Enter New Name: Yuvaraj
Update Successful.
```

```
        void computeTotal() {
            total=0;
            for (int i=0; i<no_of_products; i++)
                total+=(products[i].price*products[i].quantity);
        }
        void displayBill() {
            computeTotal();
            cout<<"\nCustomer Phone No: "<<customer.phone_no<<endl;
            cout<<"Customer Name: "<<customer.name<<endl;
            cout<<"\nNo.\tProduct_Name\tUnit_Price\tQuantity\tTotal_Price
\n" <<endl;
            for (int i=0; i<no_of_products; i++)
                cout<< i+1 << "\t" << products[i].product_name << "\t\t"
<<products[i].price<<"\t\t"<<products[i].quantity<<"\t\t"<<products[i].price*pr
oducts[i].quantity<<endl;
            cout<<"\nTotal Amount = "<<total<<endl;
        }
        ~Bill() {
            cout<<"Bill Details Processed."<<endl;
        }
};
int searchCustomer (Customer * cs, int total, string p_no) {
    for (int i=0; i<total; i++) {
        if (cs[i].phone_no==p_no)
            return i;
    }
    return -1;
}
int searchProduct (Product * ps, int total, int p_id) {
    for (int i=0; i<total; i++)
        if (ps[i].product_id==p_id)
            return i;
    return -1;
}
int searchBill (Bill * bs, int total, int b_id) {
    for (int i=0; i<total; i++)
        if (bs[i].bill_id==b_id)
            return i;
    return -1;
}
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 6

Product Name (space and enter for no name): Apple
Price (non-positive value to give default price): 30
Quantity (10 if previous two values are default): 300

Apple successfully added.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 6

Product Name (space and enter for no name): Mango
Price (non-positive value to give default price): 40
Quantity (10 if previous two values are default): 100

Mango successfully added.
```

supermarket.cpp

```
#include <iostream>
#include <string>
#include "supermarket_class.h"
using namespace std;

int main() {
	int choice, pos, cpos, ppos;

	string Old, New;
	Customer * customers=new Customer[3];
	int no_of_customers=0;

	string name, temp;
	float price;
	int p_id=1, id, q;
	Product * products=new Product[3];
	int no_of_products=0;

	int b_id=1;
	string cphone;
	int pr_id, qty;
	Bill * bills=new Bill[3];
	int no_of_bills=0;

	while (1) {
		cout<<"\n0. Exit\n1. Add Customer\n2. Update Customer Phone
Number\n3. Update Customer Name\n4. Display Customer Details\n5. Delete
All Customers\n6. Add Item\n7. Update Item Stock\n8. Display Item
Details\n9. Create Bill\n10. Update Existing Bill\n11. Generate Bill Details\n";
		cout<<"Your Choice: ";
		cin>>choice;

		if (choice==0)
			break;
		switch (choice) {
			case 1:
				if (no_of_customers<3) {
					Customer c;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 6

Product Name (space and enter for no name): Apple
Price (non-positive value to give default price): 30
Quantity (10 if previous two values are default): 300

Apple successfully added.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 6

Product Name (space and enter for no name): Mango
Price (non-positive value to give default price): 40
Quantity (10 if previous two values are default): 100

Mango successfully added.
Product ID: 2
```

```
            if (c.getCustomerDetails()) {
                cout<<"Customer Added."<<endl;
                customers[no_of_customers]=c;
                no_of_customers++;
            }
        } else
            cout<<"Maximum Customer Count Reached."<<endl;
        break;
    case 2:
        cout<<"\nEnter Old Phone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0) {
            cout<<"Enter New Phone Number: ";
            cin>>New;
            if (customers[pos].updatePhoneNo(New))
                cout<<"Update Successful."<<endl;
            else
                cout<<"Update Failed."<<endl;
        } else
            cout<<"Phone Number not found."<<endl;
        break;
    case 3:
        cout<<"\nPhone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0) {
            getline(cin, temp);
            cout<<"Enter New Name: ";
            getline(cin, New);
            customers[pos].updateName(New);
            cout<<"Update Successful."<<endl;
        } else
            cout<<"Phone Number not found."<<endl;
        break;
    case 4:
        cout<<"\nPhone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0)
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 6

Product Name (space and enter for no name): Banana
Price (non-positive value to give default price): 25
Quantity (10 if previous two values are default): 100

Banana successfully added.
Product ID: 3

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 7

Product ID: 1
Quantity: 25

Quantity of Apple successfully updated.
New Quantity = 325
```

```
                        customers[pos].displayCustomerDetails();
                    else
                        cout<<"Customer not found."<<endl;
                    break;
                case 5:
                    no_of_customers=0;
                    cout<<"\nAll customers deleted."<<endl;
                    break;
                case 6:
                    if (no_of_products<3) {
                        cout<<"\nProduct Name (space and enter for no name):
";
                        getline(cin, temp);
                        getline(cin, name);
                        cout<<"Price (non-positive value to give default price):
";
                        cin>>price;
                        cout<<"Quantity (10 if previous two values are
default): ";
                        cin>>q;
                        if (name==" " && price<=0) {
                            Product p(p_id);
                            products[no_of_products]=p;
                        } else if (name!=" " && price<=0) {
                            Product p(p_id, name, q);
                            products[no_of_products]=p;
                        } else if (name==" " && price>0) {
                            Product p(p_id, price, q);
                            products[no_of_products]=p;
                        } else if (name!=" " && price>0) {
                            Product p(p_id, price, name, q);
                            products[no_of_products]=p;
                        }
                        no_of_products++;
                        cout<<"\n"<<name<<" successfully added."<<endl;
                        cout<<"Product ID: "<<p_id<<endl;
                        p_id++;
                    } else
                        cout<<"Maximum Product Count Reached."<<endl;
                    break;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 8

Product ID: 2

Product Name: Mango
Price: 40
Available Quantity: 100

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 9

Customer Phone Number: 8939314400
Product ID: 1
Quantity: 2
2 units of Apple has been added.
Bill Created
Bill ID: 1
Bill Details Processed.
```

```
            case 7:
                cout<<"\nProduct ID: ";
                cin>>id;
                pos=searchProduct(products, no_of_products, id);
                if (pos>=0) {
                    cout<<"Quantity: ";
                    cin>>q;
                    products[pos].updateStock(q);
                } else
                    cout<<"Product ID not found."<<endl;
                break;
            case 8:
                cout<<"\nProduct ID: ";
                cin>>id;
                pos=searchProduct(products, no_of_products, id);
                if (pos>=0)
                    products[pos].displayProductDetails();
                else
                    cout<<"Product ID not found."<<endl;
                break;
            case 9:
                cout<<"\nCustomer Phone Number: ";
                cin>>cphone;
                cpos=searchCustomer(customers, no_of_customers,
cphone);
                if (cpos>=0) {
                    cout<<"Product ID: ";
                    cin>>pr_id;
                    ppos=searchProduct(products, no_of_products, pr_id);
                    if (ppos>=0) {
                        cout<<"Quantity: ";
                        cin>>qty;
                        Product p=products[ppos];
                        if (qty<=p.quantity) {
                            p.quantity=qty;
                            products[ppos].quantity-=qty;

                            Bill b;
                            b.createBill(b_id);
                            b_id++;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 10

Bill ID: 1
Product ID: 2
Quantity: 3
3 units of Mango has been added.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 10

Bill ID: 1
Product ID: 3
Quantity: 4
4 units of Banana has been added.
```

```
                                        b.getCustomer(customers[cpos]);
                                        b.addProduct(p);
                                        cout<<"Bill Created\nBill ID:
"<<b.bill_id<<endl;
                                        bills[no_of_bills]=b;
                                        no_of_bills++;
                                    } else
                                        cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                                } else
                                    cout<<"Product not found."<<endl;
                            } else
                                cout<<"Customer not found."<<endl;
                            break;
                        case 10:
                            cout<<"\nBill ID: ";
                            cin>>id;
                            pos=searchBill(bills, no_of_bills, id);
                            if (pos>=0) {
                                cout<<"Product ID: ";
                                cin>>pr_id;
                                ppos=searchProduct(products, no_of_products, pr_id);
                                if (ppos>=0) {
                                    cout<<"Quantity: ";
                                    cin>>qty;
                                    Product p=products[ppos];
                                    if (qty<=p.quantity) {
                                        p.quantity=qty;
                                        products[ppos].quantity-=qty;
                                        bills[pos].addProduct(p);
                                    } else
                                        cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                                } else
                                    cout<<"Product ID not found."<<endl;
                            } else
                                cout<<"Bill ID not found."<<endl;
                            break;
                        case 11:
                            cout<<"\nBill ID: ";
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 11

Bill ID: 1

Customer Phone No: 8939314400
Customer Name: Yuvaraj

No.     Product_Name    Unit_Price      Quantity        Total_Price

1       Apple           30              2               60
2       Mango           40              3               120
3       Banana          25              4               100

Total Amount = 280

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Item
7. Update Item Stock
8. Display Item Details
9. Create Bill
10. Update Existing Bill
11. Generate Bill Details
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_01$
```

```
                cin>>id;
                pos=searchBill(bills, no_of_bills, id);
                if (pos>=0)
                    bills[pos].displayBill();
                else
                    cout<<"Bill ID not found."<<endl;
                break;
            default:
                cout<<"Invalid Choice."<<endl;
                break;
        }
    }
    return 0;
}
```

RESULT

A supermarket management application system was successfully implemented using classes, objects, constructors, destructors, and string handling.

Exp. No: 2
Date: 20/02/2024

IMPLEMENTATION OF PROGRAMS USING FUNCTION OVERLOADING AND OPERATING OVERLOADING

AIM

To implement programs using function overloading and operator overloading for the supermarket management application system.

ALGORITHM

Function Overloading is a polymorphism feature in Object Oriented Programming, where 2 or more functions can have the same name, but different parameters.

Add an update function in **Product** class that can update the product name or price or quantity, based on the parameter passed.

```
update (newname) {
        Set name of the product as newname
        Display old name and new name
}
update (price) {
        Set product price to price
        Display old price and new price
}
update (quantity) {
        Add quantity to existing quantity of product
        Display old quantity and updated quantity
}
```

Operator Overloading is a compile-time polymorphism feature; it is an idea of giving special meaning to an existing operator without changing its original meaning.

Use + operator to add 2 products, under the following conditions:

- Product names should be the same.
- Lowercase and uppercase constraints are ignored.
    - *p1: apple* and *p2: Apple* are considered a match

```
operator + (p2) {
        result;
        result=this product + p2;
        return result;
}
```

Use + operator to combine 2 bills which are under the same customer. The following conditions must hold:

- Customers should match.
- If the bills have the same products, they should be combined.
- Total no. of products after combination should remain within the upper limit, otherwise, the operation shouldn't be performed.

```
operator + (b2) {
        result;
        if customer name of this bill == customer name of b2 then
                Cycle through every product i of this bill
                        Cycle through every product j of b2
                                if i == j
                                        combine into result
        return result;
}
```

SOURCE CODE

supermarket_class.h

```
#include <iostream>
#include <string>
using namespace std;
class Customer {
	public:
		string phone_no;
		string name;
		string temp;
		int isValid(string pno) {
			if (pno.length()!=10) {
				cout<<"Invalid Phone Number."<<endl;
				return 0;
			}
			return 1;
		}
		int getCustomerDetails() {
			getline(cin, temp);
			cout<<"\nPhone Number: ";
			getline(cin, phone_no);
			if (isValid(phone_no)) {
				cout<<"Name: ";
				getline(cin, name);
				return 1;
			}
			return 0;
		}
		int updatePhoneNo (string newno) {
			if (isValid(newno)) {
				phone_no=newno;
				return 1;
			}
			return 0;
		}
		void updateName (string newname) {
			name=newname;
		}
```

```
        void displayCustomerDetails() {
                cout<<"\nPhone No: "<<phone_no<<endl;
                cout<<"Customer Name: "<<name<<endl;
            }
};

class Product {
    public:
        int product_id, quantity;
        float price;
        string product_name;
        Product() {}
        Product(int id) {
            product_id=id;
            product_name="Unnamed";
            price=500;
            quantity=10;
        }
        Product (int id, float p, int q) {
            product_id=id;
            product_name="Unnamed";
            price=p;
            quantity=q;
        }
        Product (int id, string name, int q) {
            product_id=id;
            product_name=name;
            cout<<product_name;
            price=500;
            quantity=q;
        }
        Product (int id, float p, string name, int q) {
            product_id=id;
            product_name=name;
            price=p;
            quantity=q;
        }
```

```
        Product operator + (Product const &p) {
            Product result;
            result.product_id=product_id;
            result.quantity=quantity+p.quantity;
            result.price=price;
            result.product_name=product_name;
            return result;
        }
        void update (string newname) {
            cout<<"\nOld Name: "<<product_name<<endl;
            product_name=newname;
            cout<<"Product Name successfully updated."<<endl;
            cout<<"New Name: "<<product_name<<endl;
        }
        void update (float p) {
            cout<<"\nOld Price = "<<price<<endl;
            price=p;
            cout<<"Price of "<<product_name<<" successfully
updated."<<endl;
            cout<<"New Price = "<<price<<endl;
        }
        void update (int q) {
            cout<<"\nOld Quantity = "<<quantity<<endl;
            quantity+=q;
            cout<<"Quantity of "<<product_name<<" successfully
updated."<<endl;
            cout<<"New Quantity = "<<quantity<<endl;
        }
        void displayProductDetails() {
            cout<<"\nProduct Name: "<<product_name<<endl;
            cout<<"Price: "<<price<<endl;
            cout<<"Available Quantity: "<<quantity<<endl;
        }
};
```

```
class Bill {
	public:
		int bill_id, no_of_products;
		Customer customer;
		Product * products=new Product[3];
		float total;
		Bill operator + (Bill const &b) {
			Bill result;
			for (int i=0; i<b.no_of_products; i++) {
				int c=1;
				for (int j=0; j<no_of_products; j++) {
					if (products[j].product_id==b.products[i].product_id) {
						products[j].quantity+=b.products[i].quantity;
						c=0;
						break;
					}
				}
				if (c) {
					products[no_of_products]=b.products[i];
					no_of_products++;
				}
			}
			result.bill_id=bill_id;
			result.customer=customer;
			result.products=products;
			result.no_of_products=no_of_products;
			return result;
		}
		void createBill (int id) {
			bill_id=id;
			no_of_products=0;
		}
		void getCustomer(Customer c) {
			customer=c;
		}
```

```
        int checkProduct(Product p) {
            for (int i=0; i<no_of_products; i++)
                if (products[i].product_id==p.product_id)
                    return i;
            return -1;
        }
        void addProduct(Product p) {
            int pos=checkProduct(p);
            if (pos>=0) {
                products[pos].quantity+=p.quantity;
                cout<<p.product_name<<" already exists. "<<p.quantity<<"
units of "<<p.product_name<<" has been added."<<endl;
            } else {
                products[no_of_products]=p;
                cout<<p.quantity<<" units of "<<p.product_name<<" has
been added."<<endl;
                no_of_products++;
            }
        }
        void computeTotal() {
            total=0;
            for (int i=0; i<no_of_products; i++)
                total+=(products[i].price*products[i].quantity);
        }
        void displayBill() {
            computeTotal();
            cout<<"\nCustomer Phone No: "<<customer.phone_no<<endl;
            cout<<"Customer Name: "<<customer.name<<endl;

cout<<"\nNo.\tProduct_Name\tUnit_Price\tQuantity\tTotal_Price\n"<<endl;
            for (int i=0; i<no_of_products; i++) {

cout<<i+1<<"\t"<<products[i].product_name<<"\t\t"<<products[i].price<<"\t\t"
<<products[i].quantity<<"\t\t"<<products[i].price*products[i].quantity<<endl;
            }
            cout<<"\nTotal Amount = "<<total<<endl;
        }
};
```

```
int searchCustomer (Customer * cs, int total, string p_no) {
	for (int i=0; i<total; i++) {
		if (cs[i].phone_no==p_no)
			return i;
	}
	return -1;
}

int searchProduct (Product * ps, int total, int p_id) {
	for (int i=0; i<total; i++)
		if (ps[i].product_id==p_id)
			return i;
	return -1;
}

int searchBill (Bill * bs, int total, int b_id) {
	for (int i=0; i<total; i++)
		if (bs[i].bill_id==b_id)
			return i;
	return -1;
}
```

supermarket.cpp

```
#include <iostream>
#include <string>
#include <cctype>
#include "supermarket_class.h"
using namespace std;

int main() {

	// General Variables
	int choice, pos, cpos, ppos;

	// Customer Variables
	string Old, New;
	Customer * customers=new Customer[3];
	int no_of_customers=0;

	// Product Variables
	int p_id=1;
	string name, temp;
	float price;
	int id, q;
	Product * products=new Product[3];
	int no_of_products=0;

	// Bill Variables
	int b_id=1;
	string cphone;
	int pr_id, qty;
	Bill * bills=new Bill[3];
	int no_of_bills=0;

	// Variables for Operator Overloading
	int o1, o2, o1pos, o2pos;
```

```
    while (1) {
        cout<<"\n0. Exit\n1. Add Customer\n2. Update Customer Phone
Number\n3. Update Customer Name\n4. Display Customer Details\n5. Delete
All Customers\n6. Add Product\n7. Change Product Name\n8. Update Product
Price\n9. Update Product Stock\n10. Combine Products\n11. Display Product
Details\n12. Create Bill\n13. Update Existing Bill\n14. Combine Bills\n15.
Generate Bill Details\n";
        cout<<"Your Choice: ";
        cin>>choice;

        if (choice==0)
            break;
        switch (choice) {
            case 1:
                if (no_of_customers<3) {
                    Customer c;
                    if (c.getCustomerDetails()) {
                        cout<<"Customer Added."<<endl;
                        customers[no_of_customers]=c;
                        no_of_customers++;
                    }
                } else
                    cout<<"Maximum Customer Count Reached."<<endl;
                break;
            case 2:
                cout<<"\nEnter Old Phone Number: ";
                cin>>Old;
                pos=searchCustomer(customers, no_of_customers, Old);
                if (pos>=0) {
                    cout<<"Enter New Phone Number: ";
                    cin>>New;
                    if (customers[pos].updatePhoneNo(New))
                        cout<<"Update Successful."<<endl;
                    else
                        cout<<"Update Failed."<<endl;
                } else
                    cout<<"Phone Number not found."<<endl;
                break;
            case 3:
                cout<<"\nPhone Number: ";
```

```
                cin>>Old;
                pos=searchCustomer(customers, no_of_customers, Old);
                if (pos>=0) {
                        getline(cin, temp);
                        cout<<"Enter New Name: ";
                        getline(cin, New);
                        customers[pos].updateName(New);
                        cout<<"Update Successful."<<endl;
                } else
                        cout<<"Phone Number not found."<<endl;
                break;
        case 4:
                cout<<"\nPhone Number: ";
                cin>>Old;
                pos=searchCustomer(customers, no_of_customers, Old);
                if (pos>=0)
                        customers[pos].displayCustomerDetails();
                else
                        cout<<"Customer not found."<<endl;
                break;
        case 5:
                no_of_customers=0;
                cout<<"\nAll customers deleted."<<endl;
                break;
        case 6:
                if (no_of_products<3) {
                        cout<<"\nProduct Name (space and enter for no name):
";
                        getline(cin, temp);
                        getline(cin, name);
                        cout<<"Price (non-positive value to give default price):
";
                        cin>>price;
                        cout<<"Quantity (10 if previous two values are
default): ";
                        cin>>q;
                        if (name==" " && price<=0) {
                                Product p(p_id);
                                products[no_of_products]=p;
                        } else if (name!=" " && price<=0) {
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_02$ g++ supermarket.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_02$ ./a.out

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 1

Phone Number: 7550099095
Name: Yuvaraj
Customer Added.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
```

```
                Product p(p_id, name, q);
                products[no_of_products]=p;
            } else if (name==" " && price>0) {
                Product p(p_id, price, q);
                products[no_of_products]=p;
            } else if (name!=" " && price>0) {
                Product p(p_id, price, name, q);
                products[no_of_products]=p;
            }
            cout<< "\n" <<
products[no_of_products].product_name<<" successfully added."<<endl;
            no_of_products++;
            cout<<"Product ID: "<<p_id<<endl;
            p_id++;
        } else
            cout<<"Maximum Product Count Reached."<<endl;
        break;
    case 7:
        cout<<"\nProduct ID: ";
        cin>>id;
        pos=searchProduct(products, no_of_products, id);
        if (pos>=0) {
            cout<<"Updated Name: ";
            cin>>name;
            products[pos].update(name);
        } else
            cout<<"Product ID not found."<<endl;

        break;
    case 8:
        cout<<"\nProduct ID: ";
        cin>>id;
        pos=searchProduct(products, no_of_products, id);
        if (pos>=0) {
            cout<<"Updated Price: ";
            cin>>price;
            products[pos].update(price);
        } else
            cout<<"Product ID not found."<<endl;
        break;
```

```
Your Choice: 6

Product Name (space and enter for no name): Apple
Price (non-positive value to give default price): 100
Quantity (10 if previous two values are default): 25

Apple successfully added.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 6

Product Name (space and enter for no name): apple
Price (non-positive value to give default price): 100
Quantity (10 if previous two values are default): 50

apple successfully added.
Product ID: 2
```

```
case 9:
        cout<<"\nProduct ID: ";
        cin>>id;
        pos=searchProduct(products, no_of_products, id);
        if (pos>=0) {
                cout<<"Quantity: ";
                cin>>q;
                products[pos].update(q);
        } else
                cout<<"Product ID not found."<<endl;
        break;
case 10:
        cout<<"\nID of First Product: ";
        cin>>o1;
        o1pos=searchProduct(products, no_of_products, o1);
        if (o1pos>=0) {
                cout<<"ID of Second Product: ";
                cin>>o2;
                o2pos=searchProduct(products, no_of_products, o2);
                if (o2pos>=0) {
                        if (o1==o2)
                                cout<<"The IDs are the same."<<endl;
                        else {
                                string n1=products[o1pos].product_name;
                                string n2=products[o2pos].product_name;
                                int l1=n1.length();
                                int l2=n2.length();
                                if (l1==l2) {
                                        int c=1;
                                        for (int i=0; i<l1; i++) {
                                            if (toupper(n1[i]) != toupper(n2[i])) {
                                                c=0;
                                                break;
                                            }
                                        }
                                        if (c) {
                                           if (o1<o2) {
                        products[o1pos]=products[o1pos]+products[o2pos];
                                                no_of_products--;
                        cout<<"Products Successfully Combined."<<endl;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 10

ID of First Product: 1
ID of Second Product: 2
Products Successfully Combined.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 12
```

```
cout<<"Product ID: "<<o1<<endl;
                                } else {
        products[o2pos]=products[o1pos]+products[o2pos];
                                  no_of_products--;
        cout<<"Products Successfully Combined."<<endl;
        cout<<"Product ID: "<<o2<<endl;
                                }
                            } else
                      cout<<"Product Names don't match."<<endl;
                       } else
                 cout<<"Product Names don't match."<<endl;
                 }
            } else
                 cout<<"ID not found."<<endl;
      } else
            cout<<"ID not found."<<endl;
      break;
case 11:
      cout<<"\nProduct ID: ";
      cin>>id;
      pos=searchProduct(products, no_of_products, id);
      if (pos>=0)
            products[pos].displayProductDetails();
      else
            cout<<"Product ID not found."<<endl;
      break;
case 12:
      cout<<"\nCustomer Phone Number: ";
      cin>>cphone;
   cpos=searchCustomer(customers, no_of_customers, cphone);
      if (cpos>=0) {
            cout<<"Product ID: ";
            cin>>pr_id;
            ppos=searchProduct(products, no_of_products, pr_id);
            if (ppos>=0) {
                  cout<<"Quantity: ";
                  cin>>qty;
                  Product p=products[ppos];
                  if (qty<=p.quantity) {
```

```
Customer Phone Number: 7550099095
Product ID: 1
Quantity: 20
20 units of Apple has been added.
Bill Created
Bill ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 12

Customer Phone Number: 7550099095
Product ID: 1
Quantity: 25
25 units of Apple has been added.
Bill Created
Bill ID: 2
```

```
                    p.quantity=qty;
                    products[ppos].quantity-=qty;

                    Bill b;
                    b.createBill(b_id);
                    b_id++;

                    b.getCustomer(customers[cpos]);
                    b.addProduct(p);
                cout<<"Bill Created\nBill ID: "<<b.bill_id<<endl;
                    bills[no_of_bills]=b;
                    no_of_bills++;
                } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
            } else
                cout<<"Product not found."<<endl;
        } else
            cout<<"Customer not found."<<endl;
        break;
    case 13:
        cout<<"\nBill ID: ";
        cin>>id;
        pos=searchBill(bills, no_of_bills, id);
        if (pos>=0) {
            cout<<"Product ID: ";
            cin>>pr_id;
            ppos=searchProduct(products, no_of_products, pr_id);
            if (ppos>=0) {
                cout<<"Quantity: ";
                cin>>qty;
                Product p=products[ppos];
                if (qty<=p.quantity) {
                    p.quantity=qty;
                    products[ppos].quantity-=qty;
                    bills[pos].addProduct(p);
                } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
            } else
                cout<<"Product ID not found."<<endl;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 14

ID of Bill 1: 1
ID of Bill 2: 2
Bills Successfully Combined.
Bill ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 15
```

```
                } else
                        cout<<"Bill ID not found."<<endl;
                break;
        case 14:
                cout<<"\nID of Bill 1: ";
                cin>>o1;
                o1pos=searchBill(bills, no_of_bills, o1);
                if (o1pos>=0) {
                        cout<<"ID of Bill 2: ";
                        cin>>o2;
                        o2pos=searchBill(bills, no_of_bills, o2);
                        if (o2pos>=0) {
                                if (o1==o2)
                                        cout<<"The IDs are the same."<<endl;
                                else {
                                        string c1=bills[o1pos].customer.phone_no;
                                        string c2=bills[o2pos].customer.phone_no;
                                        if (c1==c2) {
                                                int np1=bills[o1pos].no_of_products;
                                                int np2=bills[o2pos].no_of_products;
                                                int total_no=np1+np2;
                                                for (int i=0; i<np1; i++) {
                                                        for (int j=0; j<np2; j++) {
if (bills[o1pos].products[i].product_id==bills[o2pos].products[j].product_id) {
                                                                total_no--;
                                                                }
                                                        }
                                                }
                                                if (total_no<=3) {
                                                        if (o1<o2) {
                                                bills[o1pos]=bills[o1pos]+bills[o2pos];
                                                                no_of_bills--;
                                                cout<<"Bills Successfully Combined."<<endl;
                                                                cout<<"Bill ID: "<<o1<<endl;
                                                        } else {
                                                bills[o2pos]=bills[o2pos]+bills[o1pos];
                                                                no_of_bills--;
                                                cout<<"Bills Successfully Combined."<<endl;
                                                        cout<<"Bill ID: "<<o2<<endl;
                                                        }
```

```
Bill ID: 1

Customer Phone No: 7550099095
Customer Name: Yuvaraj

No.     Product_Name    Unit_Price      Quantity        Total_Price

1       Apple           100             45              4500

Total Amount = 4500

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Change Product Name
8. Update Product Price
9. Update Product Stock
10. Combine Products
11. Display Product Details
12. Create Bill
13. Update Existing Bill
14. Combine Bills
15. Generate Bill Details
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_02$
```

```
                                        } else
                                                cout<<"These bills can't be
combined as the product count exceeds the assigned limit."<<endl;
                                    } else
                                        cout<<"These bills can't be combined as
they belong to different customers."<<endl;

                                }
                            } else
                                cout<<"ID not found."<<endl;
                        } else
                            cout<<"ID not found."<<endl;

                        break;
                    case 15:
                        cout<<"\nBill ID: ";
                        cin>>id;
                        pos=searchBill(bills, no_of_bills, id);
                        if (pos>=0)
                            bills[pos].displayBill();
                        else
                            cout<<"Bill ID not found."<<endl;
                        break;
                    default:
                        cout<<"Invalid Choice."<<endl;
                        break;
                }
            }
            return 0;
}
```

RESULT

Function overloading and operator overloading were successfully implemented in the supermarket management system application.

Exp. No: 3
27/02/2024

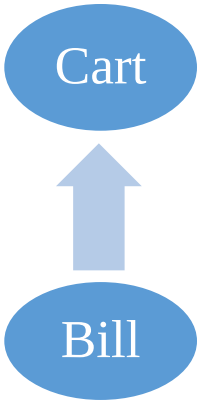
IMPLEMENTATION OF AN APPLICATIN USING INHERITANCE, VIRTUAL FUNCTIONS, AND ABSTRACT CLASSES

AIM

To implement the supermarket management system application using different types of inheritance, virtual functions, and abstract classes.

ALGORITHM

1) **Single Inheritance**

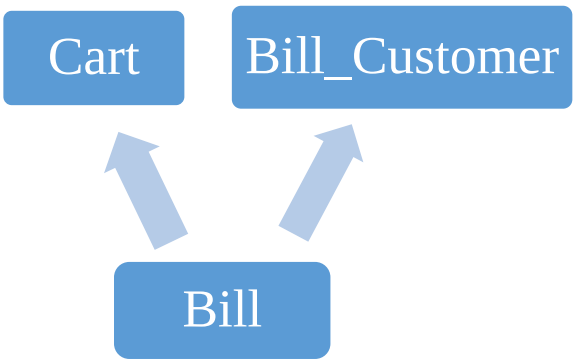



```
class Cart {
        Add Products or Items to Cart
};

class Bill: public Cart {
        Create Bill
        Get Customer Details
        Display Bill
};
```

2) Multilevel Inheritance

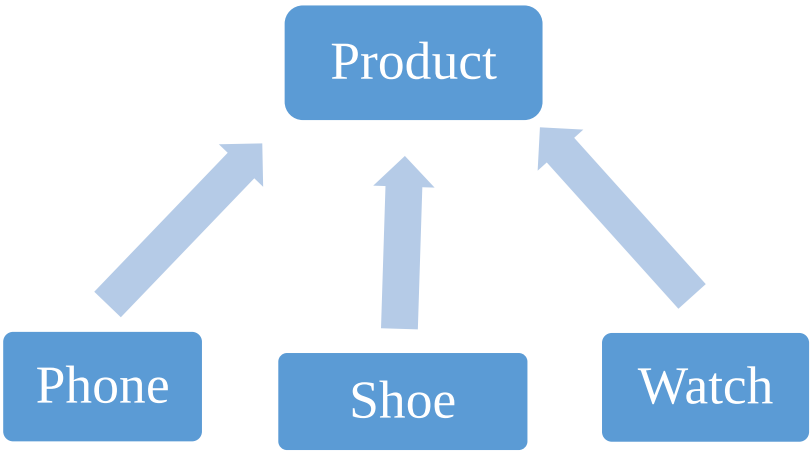
```
class Phone: public Product {
        software, battery, processor;
        Get Details
        Display
}

Class iPhone: public Phone {
        notch, unlocking method, accessories;
        Get Details
        Display
}
```

3) Multiple Inheritance




```
class Cart {
    Add Products or Items to Cart
};

class Bill_Customer {
    Add Customer existing in database to Bill
}

class Bill: public Cart, public Customer {
    Create Bill
    Update Bill
    Display Bill
}
```

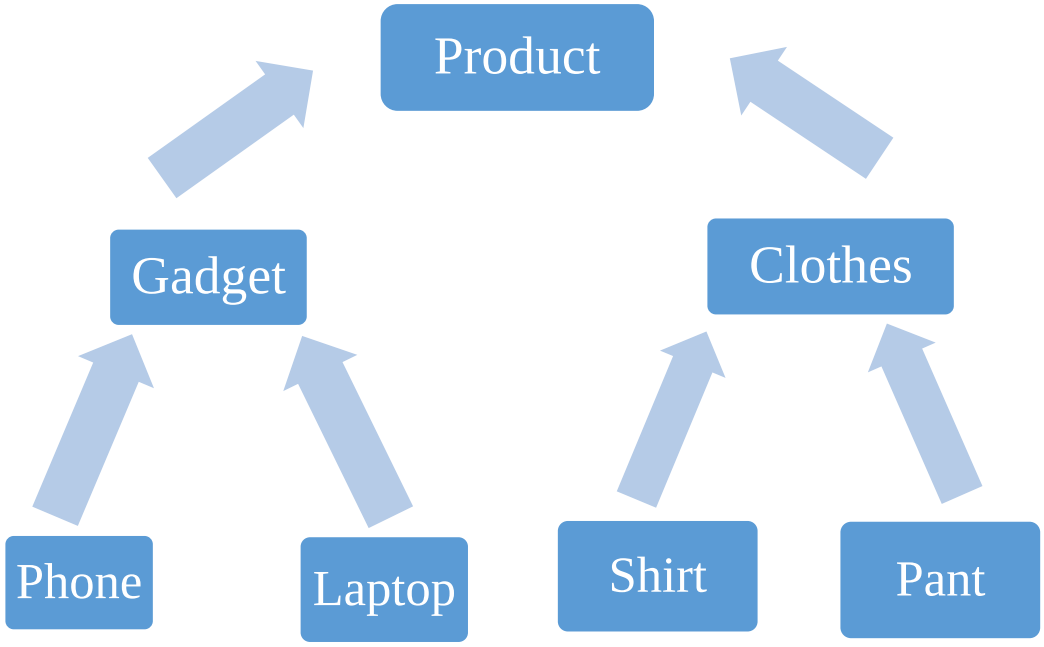
4) Hierarchical Inheritance




```
class Phone: public Product {
    software, battery, processor;
    Get Details
    Display
}

class Shoe: public Product {
    brand, type, size;
    Get Details
    Display
}

class Watch: public Product {
    size, battery, water_resistance;
    Get Details
    Display
}
```

5) Hybrid Inheritance

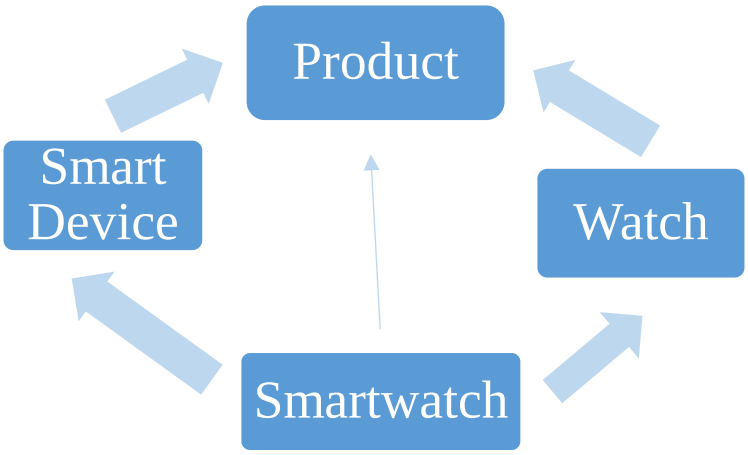
Multilevel x Hierarchical




```
class Gadget: public Product {
    brand, display, battery;
    Get Details
    Display
};
class Phone: public Gadget {
    unlock_methods, connection, cameras;
    Get Details
    Display
};
class Laptop: public Gadget {
    fans, trackpad, keyboard;
    Get Details
    Display
};
```

Similarly, define child class Clothes and grandchild classes Shirt and Pant.

6) Multipath Inheritance




```
class SmartDevice: public Product {
    software, display;
    Get Details
    Display
}

class Watch: public Product {
    dial, shape, type;
    Get Details
    Display
}

class SmartWatch: public SmartDevice, public Watch {
    pedometer, heart rate, spo2;
    Get Details
    Display
}
```

7) Virtual Function and Abstract Class

```
class Product {
    virtual getDetails();
    virtual display()=0;
}

class Phone: public Product {
    getDetails();
    display();
}

Class Shoe: public Product {
    getDetails();
    display();
}
```

SOURCE CODE

Inheritance

supermarket_class.h

```
#include <iostream>
#include <string>
using namespace std;

class Customer {
	public:
		string phone_no;
		string name;
		string temp;
		int isValid(string pno) {
			if (pno.length()!=10) {
				cout<<"Invalid Phone Number."<<endl;
				return 0;
			}
			return 1;
		}
		int getCustomerDetails() {
			getline(cin, temp);
			cout<<"\nPhone Number: ";
			getline(cin, phone_no);
			if (isValid(phone_no)) {
				cout<<"Name: ";
				getline(cin, name);
				return 1;
			}
			return 0;
		}
		int updatePhoneNo (string newno) {
			if (isValid(newno)) {
				phone_no=newno;
				return 1;
			}
			return 0;
		}
```

```
        void updateName (string newname) {
            name=newname;
        }
        void displayCustomerDetails() {
            cout<<"\nPhone No: "<<phone_no<<endl;
            cout<<"Customer Name: "<<name<<endl;
        }
};

class Product {
    public:
        int product_id, quantity;
        float price;
        string product_name;
        void getProductDetails (int id, float p, string name, int q) {
            product_id=id;
            product_name=name;
            price=p;
            quantity=q;
        }
        void update (string newname) {
            cout<<"\nOld Name: "<<product_name<<endl;
            product_name=newname;
            cout<<"Product Name successfully updated."<<endl;
            cout<<"New Name: "<<product_name<<endl;
        }
        void update (float p) {
            cout<<"\nOld Price = "<<price<<endl;
            price=p;
            cout<<"Price of "<<product_name<<" successfully
updated."<<endl;
            cout<<"New Price = "<<price<<endl;
        }
        void update (int q) {
            cout<<"\nOld Quantity = "<<quantity<<endl;
            quantity+=q;
            cout<<"Quantity of "<<product_name<<" successfully
updated."<<endl;
            cout<<"New Quantity = "<<quantity<<endl;
        }
```

```
        void displayProductDetails() {
            cout<<"\nProduct Name: "<<product_name<<endl;
            cout<<"Price: "<<price<<endl;
            cout<<"Available Quantity: "<<quantity<<endl;
        }
};

class Gadget: public Product {
    public:
        string brand, display;
        int battery;
        void getGadgetDetails(string b, string d, int bat) {
            brand=b;
            display=d;
            battery=bat;
        }
        void displayGadgetDetails() {
            cout<<"Brand: "<<brand<<endl;
            cout<<"Display Technology: "<<display<<endl;
            cout<<"Battery Capacity: "<<battery<<endl;
        }
};

class Phone: public Gadget {
    public:
        int cameras, unlock;
        string connection;
        void getPhoneDetails(int c, int u, string con) {
            cameras=c;
            unlock=u;
            connection=con;
        }
        void displayPhoneDetails() {
            cout<<"No. of Cameras: "<<cameras<<endl;
            cout<<"No. of Unlock Methods: "<<unlock<<endl;
            cout<<"Connection: "<<connection<<endl;
        }
};
```

```
class Laptop: public Gadget {
	public:
		string fans, trackpad, keyboard;
		void getLaptopDetails(string f, string t, string k) {
			fans=f;
			trackpad=t;
			keyboard=k;
		}
		void displayLaptopDetails() {
			cout<<"Fans: "<<fans<<endl;
			cout<<"Trackpad: "<<trackpad<<endl;
			cout<<"Keyboard: "<<keyboard<<endl;
		}
};
class Clothes: public Product {
	public:
		string material, type;
		float size;
		void getClothDetails	(string m, string t, float s) {
			material=m;
			type=t;
			size=s;
		}
		void displayClothDetails () {
			cout<<"Material: "<<material<<endl;
			cout<<"Type: "<<type<<endl;
			cout<<"Cloth Size: "<<size<<endl;
		}
};
class Shirt: public Clothes {
	public:
		string buttons, length;
		void getShirtDetails(string b, string l) {
			buttons=b;
			length=l;
		}
		void displayShirtDetails() {
			cout<<"Button Type: "<<buttons<<endl;
			cout<<"Arm-Length: "<<length<<endl;
		}};
```

```
class Pant: public Clothes {
	public:
		int pockets;
		string button;
		void getPantDetails(int p, string b) {
			pockets=p;
			button=b;
		}
		void displayPantDetails() {
			cout<<"No. of Pockets: "<<pockets<<endl;
			cout<<"Pant Button Type: "<<button<<endl;
		}

};

class Cart {
	public:
		Phone * phones=new Phone[3];
		Laptop * laptops=new Laptop[3];
		Shirt * shirts=new Shirt[3];
		Pant* pants=new Pant[3];
		int no_of_phones, no_of_laptops;
		int no_of_shirts, no_of_pants;
		int checkPhone(Phone p) {
			for (int i=0; i<no_of_phones; i++)
				if (phones[i].product_id==p.product_id)
					return i;
			return -1;
		}
		void addPhone(Phone p) {
			int pos=checkPhone(p);
			if (pos>=0) {
				phones[pos].quantity+=p.quantity;
				cout<<p.product_name<<" already exists. "<<p.quantity<<"
units of "<<p.product_name<<" has been added."<<endl;
			} else {
				phones[no_of_phones]=p;
				cout<<p.quantity<<" units of "<<p.product_name<<" has
been added."<<endl;
				no_of_phones++;
```

```
		}
	}
	int checkLaptop(Laptop p) {
		for (int i=0; i<no_of_laptops; i++)
			if (laptops[i].product_id==p.product_id)
				return i;
		return -1;
	}
	void addLaptop(Laptop p) {
		int pos=checkLaptop(p);
		if (pos>=0) {
			laptops[pos].quantity+=p.quantity;
			cout<<p.product_name<<" already exists. "<<p.quantity<<"
units of "<<p.product_name<<" has been added."<<endl;
		} else {
			laptops[no_of_laptops]=p;
			cout<<p.quantity<<" units of "<<p.product_name<<" has
been added."<<endl;
			no_of_laptops++;
		}
	}
	int checkShirt(Shirt p) {
		for (int i=0; i<no_of_shirts; i++)
			if (shirts[i].product_id==p.product_id)
				return i;
		return -1;
	}
	void addShirt(Shirt p) {
		int pos=checkShirt(p);
		if (pos>=0) {
			shirts[pos].quantity+=p.quantity;
			cout<<p.product_name<<" already exists. "<<p.quantity<<"
units of "<<p.product_name<<" has been added."<<endl;
		} else {
			shirts[no_of_shirts]=p;
			cout<<p.quantity<<" units of "<<p.product_name<<" has
been added."<<endl;
			no_of_shirts++;
		}
	}
```

```
        int checkPant(Pant p) {
            for (int i=0; i<no_of_pants; i++)
                if (pants[i].product_id==p.product_id)
                    return i;
            return -1;
        }
        void addPant(Pant p) {
            int pos=checkPant(p);
            if (pos>=0) {
                pants[pos].quantity+=p.quantity;
                cout<<p.product_name<<" already exists. "<<p.quantity<<"
units of "<<p.product_name<<" has been added."<<endl;
            } else {
                pants[no_of_pants]=p;
                cout<<p.quantity<<" units of "<<p.product_name<<" has
been added."<<endl;
                no_of_pants++;
            }
        }
};

class Bill: public Cart {
    public:
        int bill_id;
        Customer customer;
        float total;
        void createBill (int id) {
            bill_id=id;
            no_of_phones=0;
            no_of_laptops=0;
            no_of_shirts=0;
            no_of_pants=0;
        }
        void getCustomer(Customer c) {
            customer=c;
        }
        void computeTotal() {
            total=0;
            for (int i=0; i<no_of_phones; i++)
                total+=(phones[i].price*phones[i].quantity);
```

```
                for (int i=0; i<no_of_laptops; i++)
                        total+=(laptops[i].price*laptops[i].quantity);
                for (int i=0; i<no_of_shirts; i++)
                        total+=(shirts[i].price*shirts[i].quantity);
                for (int i=0; i<no_of_pants; i++)
                        total+=(pants[i].price*pants[i].quantity);
        }
        void displayBill() {
                computeTotal();
                cout<<"\nCustomer Phone No: "<<customer.phone_no<<endl;
                cout<<"Customer Name: "<<customer.name<<endl;

cout<<"\nNo.\tProduct_Name\tUnit_Price\tQuantity\tTotal_Price\n"<<endl;
                for (int i=0; i<no_of_phones; i++) {

cout<<i+1<<"\t"<<phones[i].product_name<<"\t"<<phones[i].price<<"\t\t"<<p
hones[i].quantity<<"\t\t"<<phones[i].price*phones[i].quantity<<endl;
                }
                for (int i=0; i<no_of_laptops; i++) {

cout<<i+1<<"\t"<<laptops[i].product_name<<"\t"<<laptops[i].price<<"\t\t"<<l
aptops[i].quantity<<"\t\t"<<laptops[i].price*laptops[i].quantity<<endl;
                }
                for (int i=0; i<no_of_shirts; i++) {

cout<<i+1<<"\t"<<shirts[i].product_name<<"\t"<<shirts[i].price<<"\t\t"<<shirt
s[i].quantity<<"\t\t"<<shirts[i].price*shirts[i].quantity<<endl;
                }
                for (int i=0; i<no_of_pants; i++) {

cout<<i+1<<"\t"<<pants[i].product_name<<"\t"<<pants[i].price<<"\t\t"<<pants
[i].quantity<<"\t\t"<<pants[i].price*pants[i].quantity<<endl;
                }
                cout<<"\nTotal Amount = "<<total<<endl;
        }
};

int searchCustomer (Customer * cs, int total, string p_no) {
        for (int i=0; i<total; i++) {
                if (cs[i].phone_no==p_no)
```

```
                return i;
        }
        return -1;
}

int searchPhone(Phone * ps, int total, int p_id) {
        for (int i=0; i<total; i++)
                if (ps[i].product_id==p_id)
                        return i;
        return -1;
}

int searchLaptop (Laptop * ps, int total, int p_id) {
        for (int i=0; i<total; i++)
                if (ps[i].product_id==p_id)
                        return i;
        return -1;
}

int searchShirt (Shirt * ps, int total, int p_id) {
        for (int i=0; i<total; i++)
                if (ps[i].product_id==p_id)
                        return i;
        return -1;
}

int searchPant (Pant * ps, int total, int p_id) {
        for (int i=0; i<total; i++)
                if (ps[i].product_id==p_id)
                        return i;
        return -1;
}

int searchBill (Bill * bs, int total, int b_id) {
        for (int i=0; i<total; i++)
                if (bs[i].bill_id==b_id)
                        return i;
        return -1;
}
```

supermarket.cpp

```
#include <iostream>
#include <string>
#include <cctype>
#include "supermarket_class.h"
using namespace std;

int main() {

        // General Variables
        int choice, pos, cpos, ppos, category;

        // Customer Variables
        string Old, New;
        Customer * customers=new Customer[3];
        int no_of_customers=0;

        // Product Variables
        int p_id=1;
        string name, temp;
        float price;
        int id, q;
        Product * products=new Product[9];
        int no_of_products=0;

        // Gadget Variables
        string brand, disp;
        int gb;

        // Phone Variables
        int cam, un;
        string con;
        Phone * phones=new Phone[9];
        int no_of_phones=0;

         // Laptop Variables
         string fan, track, kb;
         Laptop * laptops=new Laptop[9];
         int no_of_laptops=0;
```

```
// Cloth Variables
string mat, type;
float size;

// Shirt Variables
string sb, len;
Shirt * shirts=new Shirt[9];
int no_of_shirts=0;

// Pant Variables
int pocket;
string pb;
Pant * pants=new Pant[9];
int no_of_pants=0;

// Bill Variables
int b_id=1;
string cphone;
int pr_id, qty;
Bill * bills=new Bill[3];
int no_of_bills=0;

while (1) {
	cout<<"\n0. Exit\n1. Add Customer\n2. Update Customer Phone
Number\n3. Update Customer Name\n4. Display Customer Details\n5. Delete
All Customers\n6. Add Product\n7. Display Product Details\n8. Create Bill\n9.
Update Existing Bill\n10. Generate Bill Details\n";
	cout<<"Your Choice: ";
	cin>>choice;

	if (choice==0)
		break;
	switch (choice) {
		case 1:
			if (no_of_customers<3) {
				Customer c;
				if (c.getCustomerDetails()) {
					cout<<"Customer Added."<<endl;
					customers[no_of_customers]=c;
					no_of_customers++;
```

```
            }
        } else
            cout<<"Maximum Customer Count Reached."<<endl;
        break;
    case 2:
        cout<<"\nEnter Old Phone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0) {
            cout<<"Enter New Phone Number: ";
            cin>>New;
            if (customers[pos].updatePhoneNo(New))
                cout<<"Update Successful."<<endl;
            else
                cout<<"Update Failed."<<endl;
        } else
            cout<<"Phone Number not found."<<endl;
        break;
    case 3:
        cout<<"\nPhone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0) {
            getline(cin, temp);
            cout<<"Enter New Name: ";
            getline(cin, New);
            customers[pos].updateName(New);
            cout<<"Update Successful."<<endl;
        } else
            cout<<"Phone Number not found."<<endl;
        break;
    case 4:
        cout<<"\nPhone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0)
            customers[pos].displayCustomerDetails();
        else
            cout<<"Customer not found."<<endl;
        break;
```

```
                case 5:
                        no_of_customers=0;
                        cout<<"\nAll customers deleted."<<endl;
                        break;
                case 6:
                        cout<<"Product Category:\n\t1. Gadget\n\t2. Cloth"<<endl;
                        cout<<"Your Choice: ";
                        cin>>category;
                        if (category<1 || category>2)
                                break;
                        if (no_of_products<10) {
                                cout<<"\nProduct Name: ";
                                getline(cin, temp);
                                getline(cin, name);
                                cout<<"Price: ";
                                cin>>price;
                                cout<<"Quantity: ";
                                cin>>q;
                                if (category==1) {
                                        cout<<"Type of Device:\n\t1. Phone\n\t2.
Laptop"<<endl;
                                        cout<<"Your Choice: ";
                                        cin>>category;
                                        if (category<1 || category>2)
                                                break;
                                        cout<<"Brand Name: ";
                                        getline(cin, temp);
                                        getline(cin, brand);
                                        cout<<"Display Technology: ";
                                        getline(cin, disp);
                                        cout<<"Battery Capacity: ";
                                        cin>>gb;
                                        if  (category==1) {
                                                Phone p;
                                                p.getProductDetails(p_id, price, name, q);
                                                p.getGadgetDetails(brand, disp, gb);
                                                cout<<"No. of Cameras: ";
                                                cin>>cam;
                                                cout<<"No. of Unlock Methods: ";
                                                cin>>un;
```

```
                                        cout<<"Network Connection: ";
                                        getline(cin, temp);
                                        getline(cin, con);
                                        p.getPhoneDetails(cam, un, con);
                                        phones[no_of_phones]=p;
                    cout<<"\n"<<phones[no_of_phones].product_name<<"
successfully added."<<endl;
                                        no_of_phones++;
                                    } else {
                                        Laptop p;
                                        p.getProductDetails(p_id, price, name, q);
                                        p.getGadgetDetails(brand, disp, gb);
                                        cout<<"Fans: ";
                                        getline(cin, temp);
                                        getline(cin, fan);
                                        cout<<"Trackpad: ";
                                        getline(cin, track);
                                        cout<<"Keyboard: ";
                                        getline(cin, kb);
                                        p.getLaptopDetails(fan, track, kb);
                                        laptops[no_of_laptops]=p;
                     cout<<"\n"<<laptops[no_of_laptops].product_name<<"
successfully added."<<endl;
                                        no_of_laptops++;
                                    }
                                } else {
                                    cout<<"Type of Cloth:\n\t1. Shirt\n\t2.
Pant"<<endl;
                                    cout<<"Your Choice: ";
                                    cin>>category;
                                    if (category<1 || category>2)
                                        break;
                                    cout<<"Material: ";
                                    getline(cin, temp);
                                    getline(cin, mat);
                                    cout<<"Type: ";
                                    getline(cin, type);
                                    cout<<"Size: ";
                                    cin>>size;
```

```
                    if  (category==1) {
                        Shirt p;
                        p.getProductDetails(p_id, price, name, q);
                        p.getClothDetails(mat, type, size);
                        cout<<"Button Type: ";
                        getline(cin, temp);
                        getline(cin, sb);
                        cout<<"Arm-Length: ";
                        getline(cin, len);
                        p.getShirtDetails(sb, len);
                        shirts[no_of_shirts]=p;
        cout<<"\n"<<shirts[no_of_shirts].product_name<<" successfully
added."<<endl;
                        no_of_shirts++;
                    } else {
                        Pant p;
                        p.getProductDetails(p_id, price, name, q);
                        p.getClothDetails(mat, type, size);
                        cout<<"No. of Pockets: ";
                        cin>>pocket;
                        cout<<"Pant Button Type: ";
                        getline(cin, temp);
                        getline(cin, pb);
                        pants[no_of_pants]=p;
      cout<<"\n"<<pants[no_of_pants].product_name<<" successfully
added."<<endl;
                        no_of_pants++;
                    }
                }
                no_of_products++;
                cout<<"Product ID: "<<p_id<<endl;
                p_id++;
            } else {
                cout<<"Maximum Product Count Reached."<<endl;
            }
            break;
```

```
                case 7:
                        cout<<"Product Category:\n\t1. Gadget\n\t2. Cloth"<<endl;
                        cout<<"Your Choice: ";
                        cin>>category;
                        if (category<1 || category>2)
                                break;
                        if (category==1) {
                                cout<<"Type of Device:\n\t1. Phone\n\t2.
Laptop"<<endl;
                                cout<<"Your Choice: ";
                                cin>>category;
                                if (category<1 || category>2)
                                        break;
                                if (category==1) {
                                        cout<<"Product ID: ";
                                        cin>>id;
                                        pos=searchPhone(phones, no_of_phones, id);
                                        if (pos>=0) {
                                                phones[pos].displayProductDetails();
                                                phones[pos].displayGadgetDetails();
                                                phones[pos].displayPhoneDetails();
                                        } else
                                                cout<<"Product ID not found."<<endl;
                                } else {
                                        cout<<"Product ID: ";
                                        cin>>id;
                                        pos=searchLaptop(laptops, no_of_laptops, id);
                                        if (pos>=0) {
                                                laptops[pos].displayProductDetails();
                                                laptops[pos].displayGadgetDetails();
                                                laptops[pos].displayLaptopDetails();
                                        } else
                                                cout<<"Product ID not found."<<endl;
                                }
                        } else {
                                cout<<"Type of Cloth:\n\t1. Shirt\n\t2. Pant"<<endl;
                                cout<<"Your Choice: ";
                                cin>>category;
                                if (category<1 || category>2)
                                        break;
```

```
                    if (category==1) {
                        cout<<"Product ID: ";
                        cin>>id;
                        pos=searchShirt(shirts, no_of_shirts, id);
                        if (pos>=0) {
                            shirts[pos].displayProductDetails();
                            shirts[pos].displayClothDetails();
                            shirts[pos].displayShirtDetails();
                        } else
                            cout<<"Product ID not found."<<endl;
                    } else {
                        cout<<"Product ID: ";
                        cin>>id;
                        pos=searchPant(pants, no_of_pants, id);
                        if (pos>=0) {
                            pants[pos].displayProductDetails();
                            pants[pos].displayClothDetails();
                            pants[pos].displayPantDetails();
                        } else
                            cout<<"Product ID not found."<<endl;
                    }
                }
                break;
            case 8:
                cout<<"\nCustomer Phone Number: ";
                cin>>cphone;
                cpos=searchCustomer(customers, no_of_customers,
cphone);
                if (cpos>=0) {
                    cout<<"Product Category:\n\t1. Gadget\n\t2.
Cloth"<<endl;
                    cout<<"Your Choice: ";
                    cin>>category;
                    if (category<1 || category>2)
                        break;
                    if (category==1) {
                        cout<<"Type of Device:\n\t1. Phone\n\t2.
Laptop"<<endl;
                        cout<<"Your Choice: ";
                        cin>>category;
```

```
                    cout<<"Product ID: ";
                    cin>>pr_id;
                    if (category<1 || category>2)
                        break;
                    if (category==1) {
                        ppos=searchPhone(phones, no_of_phones,
pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Phone p = phones[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                phones[ppos].quantity-=qty;
                                Bill b;
                                b.createBill(b_id);
                                b_id++;
                                b.getCustomer(customers[cpos]);
                                b.addPhone(p);
                                cout<<"Bill Created\nBill ID:
"<<b.bill_id<<endl;
                                bills[no_of_bills]=b;
                                no_of_bills++;
                            } else
                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                        } else
                            cout<<"Product not found."<<endl;
                    } else {
                        ppos=searchLaptop(laptops, no_of_laptops,
pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Laptop p=laptops[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                laptops[ppos].quantity-=qty;
                                Bill b;
                                b.createBill(b_id);
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_03/final$ g++ supermarket.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_03/final$ ./a.out

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 1

Phone Number: 7550099095
Name: Yuvaraj K
Customer Added.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 6
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
```

```
                                        b_id++;
                                        b.getCustomer(customers[cpos]);
                                        b.addLaptop(p);
                                cout<<"Bill Created\nBill ID: "<<b.bill_id<<endl;
                                        bills[no_of_bills]=b;
                                        no_of_bills++;
                                    } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                                } else
                                    cout<<"Product not found."<<endl;
                            }
                        } else {
                            cout<<"Type of Cloth:\n\t1. Shirt\n\t2.
Pant"<<endl;
                            cout<<"Your Choice: ";
                            cin>>category;
                            if (category<1 || category>2)
                                break;
                            cout<<"Product ID: ";
                            cin>>pr_id;
                            if (category==1) {
                                ppos=searchShirt(shirts, no_of_shirts, pr_id);
                                if (ppos>=0) {
                                    cout<<"Quantity: ";
                                    cin>>qty;
                                    Shirt p=shirts[ppos];
                                    if (qty<=p.quantity) {
                                        p.quantity=qty;
                                        shirts[ppos].quantity-=qty;
                                        Bill b;
                                        b.createBill(b_id);
                                        b_id++;
                                        b.getCustomer(customers[cpos]);
                                        b.addShirt(p);
                                        cout<<"Bill Created\nBill ID:
"<<b.bill_id<<endl;
                                        bills[no_of_bills]=b;
                                        no_of_bills++;
                                    } else
```

```
Product Name: S24 Ultra
Price: 1200
Quantity: 25
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Brand Name: Samsung Galaxy
Display Technology: OLED
Battery Capacity: 5000
No. of Cameras: 5
No. of Unlock Methods: 3
Network Connection: 5G

S24 Ultra successfully added.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 6
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 2
```

```
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                                        } else
                                                cout<<"Product not found."<<endl;
                                        } else {
                                                ppos=searchPant(pants, no_of_pants, pr_id);
                                                if (ppos>=0) {
                                                        cout<<"Quantity: ";
                                                        cin>>qty;
                                                        Pant p=pants[ppos];
                                                        if (qty<=p.quantity) {
                                                                p.quantity=qty;
                                                                pants[ppos].quantity-=qty;
                                                                Bill b;
                                                                b.createBill(b_id);
                                                                b_id++;
                                                                b.getCustomer(customers[cpos]);
                                                                b.addPant(p);
                                                                cout<<"Bill Created\nBill ID:
"<<b.bill_id<<endl;
                                                                bills[no_of_bills]=b;
                                                                no_of_bills++;
                                                        } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                                                } else
                                                        cout<<"Product not found."<<endl;
                                        }
                                }
                        } else
                                cout<<"Customer not found."<<endl;
                        break;
                case 9:
                        cout<<"\nBill ID: ";
                        cin>>id;
                        pos=searchBill(bills, no_of_bills, id);
                        if (pos>=0) {
                                cout<<"Product Category:\n\t1. Gadget\n\t2.
Cloth"<<endl;
                                cout<<"Your Choice: ";
                                cin>>category;
                                if (category<1 || category>2)
```

```
Product Name: Liverpool 23/24
Price: 25
Quantity: 100
Type of Cloth:
        1. Shirt
        2. Pant
Your Choice: 1
Material: Cotton + Synthetic
Type: Jersey
Size: 39
Button Type: N/A
Arm-Length: Half-Sleeve

Liverpool 23/24 successfully added.
Product ID: 2

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 8
```

```
                    break;
                if (category==1) {
                    cout<<"Type of Device:\n\t1. Phone\n\t2.
Laptop"<<endl;
                    cout<<"Your Choice: ";
                    cin>>category;
                    cout<<"Product ID: ";
                    cin>>pr_id;
                    if (category<1 || category>2)
                        break;
                     if (category==1) {
                        ppos=searchPhone(phones, no_of_phones,
pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Phone p = phones[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                phones[ppos].quantity-=qty;
                                bills[pos].addPhone(p);
                            } else
                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                        } else
                            cout<<"Product not found."<<endl;
                    } else {
                        ppos=searchLaptop(laptops, no_of_laptops,
pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Laptop p=laptops[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                laptops[ppos].quantity-=qty;
                                bills[pos].addLaptop(p);
                            } else
                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
```

```
Customer Phone Number: 7550099095
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Product ID: 1
Quantity: 1
1 units of S24 Ultra has been added.
Bill Created
Bill ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 9

Bill ID: 1
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 2
Type of Cloth:
        1. Shirt
        2. Pant
Your Choice: 1
Product ID: 2
Quantity: 3
3 units of Liverpool 23/24 has been added.
```

```
                        } else
                            cout<<"Product not found."<<endl;
                    }
                } else {
                    cout<<"Type of Cloth:\n\t1. Shirt\n\t2.
Pant"<<endl;
                    cout<<"Your Choice: ";
                    cin>>category;
                    if (category<1 || category>2)
                        break;
                    cout<<"Product ID: ";
                    cin>>pr_id;
                    if (category==1) {
                        ppos=searchShirt(shirts, no_of_shirts, pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Shirt p=shirts[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                shirts[ppos].quantity-=qty;
                                bills[pos].addShirt(p);
                            } else
                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                        } else
                            cout<<"Product not found."<<endl;
                    } else {
                        ppos=searchPant(pants, no_of_pants, pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Pant p=pants[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                pants[ppos].quantity-=qty;
                                bills[pos].addPant(p);
                            } else
                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 10

Bill ID: 1

Customer Phone No: 7550099095
Customer Name: Yuvaraj K

No.     Product_Name    Unit_Price      Quantity        Total_Price

1       S24 Ultra       1200            1               1200
1       Liverpool 23/24 25              3               75

Total Amount = 1275

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_03/final$ coproc gedit supermarket_class.h
```

```
                        } else
                            cout<<"Product not found."<<endl;
                    }
                }
            } else
                cout<<"Bill ID not found."<<endl;
            break;
        case 10:
            cout<<"\nBill ID: ";
            cin>>id;
            pos=searchBill(bills, no_of_bills, id);
            if (pos>=0)
                bills[pos].displayBill();
            else
                cout<<"Bill ID not found."<<endl;
            break;
        default:
            cout<<"Invalid Choice."<<endl;
            break;
    }
}
return 0;
}
```

Virtual Functions and Abstract Classes

supermarket_class.h

```
#include <iostream>
#include <string>
using namespace std;
class Customer {
	public:
		string phone_no;
		string name;
		string temp;
		int isValid(string pno) {
			if (pno.length()!=10) {
				cout<<"Invalid Phone Number."<<endl;
				return 0;
			}
			return 1;
		}
		int getCustomerDetails() {
			getline(cin, temp);
			cout<<"\nPhone Number: ";
			getline(cin, phone_no);
			if (isValid(phone_no)) {
				cout<<"Name: ";
				getline(cin, name);
				return 1;
			}
			return 0;
		}
		int updatePhoneNo (string newno) {
			if (isValid(newno)) {
				phone_no=newno;
				return 1;
			}
			return 0;
		}
		void updateName (string newname) {
			name=newname;
		}
```

```
		void displayCustomerDetails() {
			cout<<"\nPhone No: "<<phone_no<<endl;
			cout<<"Customer Name: "<<name<<endl;
		}
};

class Product {
	public:
		int product_id, quantity;
		float price;
		string product_name;
		virtual void getDetails (int id, float p, string name, int q) {
			product_id=id;
			product_name=name;
			price=p;
			quantity=q;
		}
		void update (string newname) {
			cout<<"\nOld Name: "<<product_name<<endl;
			product_name=newname;
			cout<<"Product Name successfully updated."<<endl;
			cout<<"New Name: "<<product_name<<endl;
		}
		void update (float p) {
			cout<<"\nOld Price = "<<price<<endl;
			price=p;
			cout<<"Price of "<<product_name<<" successfully
updated."<<endl;
			cout<<"New Price = "<<price<<endl;
		}
		void update (int q) {
			cout<<"\nOld Quantity = "<<quantity<<endl;
			quantity+=q;
			cout<<"Quantity of "<<product_name<<" successfully
updated."<<endl;
			cout<<"New Quantity = "<<quantity<<endl;
		}
		virtual void display()=0;
};
```

```
class Phone: public Product {
        public:
                string software, processor;
                int battery;
                void getDetails (string s, string p, int b) {
                        software=s;
                        processor=p;
                        battery=b;
                }
                void display() {
                        cout<<"\nProduct Name: "<<product_name<<endl;
                        cout<<"Price: "<<price<<endl;
                        cout<<"Available Quantity: "<<quantity<<endl;
                        cout<<"Software Version: "<<software<<endl;
                        cout<<"Processor: "<<processor<<endl;
                        cout<<"Battery Capacity = "<<battery<<" mAh"<<endl;
                }
};

class Shoe: public Product {
        public:
                string brand, type;
                int size;
                void getDetails(string b, string t, int s) {
                        brand=b;
                        type=t;
                        size=s;
                }
                void display() {
                        cout<<"\nProduct Name: "<<product_name<<endl;
                        cout<<"Price: "<<price<<endl;
                        cout<<"Available Quantity: "<<quantity<<endl;
                        cout<<"Brand Name: "<<brand<<endl;
                        cout<<"Shoe Type: "<<type<<endl;
                        cout<<"Size: "<<size<<endl;
                }
};
```

```
class Bill {
	public:
		int bill_id;
		Customer customer;
		int no_of_phones, no_of_shoes;
		Phone * phones=new Phone[3];
		Shoe * shoes=new Shoe[3];
		float total;
		void createBill (int id) {
			bill_id=id;
			no_of_phones=0;
			no_of_shoes=0;
		}
		void getCustomer(Customer c) {
			customer=c;
		}
		int checkPhone(Phone p) {
			for (int i=0; i<no_of_phones; i++)
				if (phones[i].product_id==p.product_id)
					return i;
			return -1;
		}
		void addPhone(Phone p) {
			int pos=checkPhone(p);
			if (pos>=0) {
				phones[pos].quantity+=p.quantity;
				cout<<p.product_name<<" already exists.
"<<p.quantity<<" units of "<<p.product_name<<" has been added."<<endl;
			} else {
				phones[no_of_phones]=p;
				cout<<p.quantity<<" units of "<<p.product_name<<"
has been added."<<endl;
				no_of_phones++;
			}
		}
		int checkShoe(Shoe p) {
			for (int i=0; i<no_of_shoes; i++)
				if (shoes[i].product_id==p.product_id)
					return i;
			return -1;	}
```

```
		void addShoe(Shoe p) {
			int pos=checkShoe(p);
			if (pos>=0) {
				shoes[pos].quantity+=p.quantity;
				cout<<p.product_name<<" already exists.
"<<p.quantity<<" units of "<<p.product_name<<" has been added."<<endl;
			} else {
				shoes[no_of_shoes]=p;
				cout<<p.quantity<<" units of "<<p.product_name<<"
has been added."<<endl;
				no_of_shoes++;
			}
		}
		void computeTotal() {
			total=0;
			for (int i=0; i<no_of_phones; i++)
				total+=(phones[i].price*phones[i].quantity);
			for (int i=0; i<no_of_shoes; i++)
				total+=(shoes[i].price*shoes[i].quantity);
		}
		void displayBill() {
			computeTotal();
		cout<<"\nCustomer Phone No: "<<customer.phone_no<<endl;
			cout<<"Customer Name: "<<customer.name<<endl;

	cout<<"\nNo.\tProduct_Name\tUnit_Price\tQuantity\tTotal_Price\n"<<en
dl;
			int i;
			for (int i=0; i<no_of_phones; i++) {
	cout<<i+1<<"\t"<<phones[i].product_name<<"\t\t"<<phones[i].price<<"\
t\t"<<phones[i].quantity<<"\t\t"<<phones[i].price*phones[i].quantity<<endl;
			}
			for (int j=0; j<no_of_shoes; j++) {
	cout<<j+i+1<<"\t"<<shoes[j].product_name<<"\t\t"<<shoes[j].price<<"\t
\t"<<shoes[j].quantity<<"\t\t"<<shoes[i].price*shoes[i].quantity<<endl;
			}
			cout<<"\nTotal Amount = "<<total<<endl;
		}
};
```

```
int searchCustomer (Customer * cs, int total, string p_no) {
	for (int i=0; i<total; i++) {
		if (cs[i].phone_no==p_no)
			return i;
	}
	return -1;
}

int searchPhone (Phone * ps, int total, int p_id) {
	for (int i=0; i<total; i++)
		if (ps[i].product_id==p_id)
			return i;
	return -1;
}

int searchShoe (Shoe * s, int total, int s_id) {
	for (int i=0; i<total; i++)
		if (s[i].product_id==s_id)
			return i;
	return -1;
}

int searchBill (Bill * bs, int total, int b_id) {
	for (int i=0; i<total; i++)
		if (bs[i].bill_id==b_id)
			return i;
	return -1;
}
```

supermarket.cpp

```
#include <iostream>
#include <string>
#include <cctype>
#include "supermarket_class.h"
using namespace std;

int main() {

    // General Variables
  int choice, pos, cpos, ppos, category;
    string temp;

    // Customer Variables
  string Old, New;
  Customer * customers=new Customer[3];
  int no_of_customers=0;

    // Product Variables
  int p_id=1;
    string name;
  float price;
  int id, q;

    // Phone Variables
    string proc, disp;
    int bat;
  Phone * phones=new Phone[3];
  int no_of_phones=0;

    // Shoe Variables
    string brand, type;
    int size;
    Shoe * shoes=new Shoe[3];
    int no_of_shoes=0;

    // Bill Variables
  int b_id=1;
  string cphone;
```

```
int pr_id, qty;
Bill * bills=new Bill[3];
int no_of_bills=0;

while (1) {
	cout<<"\n0. Exit\n1. Add Customer\n2. Update Customer Phone
Number\n3. Update Customer Name\n4. Display Customer Details\n5. Delete
All Customers\n6. Add Product\n7. Display Product Details\n8. Create Bill\n9.
Update Existing Bill\n10. Generate Bill Details\n";
	cout<<"Your Choice: ";
	cin>>choice;

	if (choice==0)
		break;
	switch (choice) {
		case 1:
			if (no_of_customers<3) {
				Customer c;
				if (c.getCustomerDetails()) {
					cout<<"Customer Added."<<endl;
					customers[no_of_customers]=c;
					no_of_customers++;
				}
			} else
				cout<<"Maximum Customer Count Reached."<<endl;
			break;
		case 2:
			cout<<"\nEnter Old Phone Number: ";
			cin>>Old;
			pos=searchCustomer(customers, no_of_customers, Old);
			if (pos>=0) {
				cout<<"Enter New Phone Number: ";
				cin>>New;
				if (customers[pos].updatePhoneNo(New))
					cout<<"Update Successful."<<endl;
				else
					cout<<"Update Failed."<<endl;
			} else
				cout<<"Phone Number not found."<<endl;
			break;
```

```
case 3:
        cout<<"\nPhone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0) {
                getline(cin, temp);
                cout<<"Enter New Name: ";
                getline(cin, New);
                customers[pos].updateName(New);
                cout<<"Update Successful."<<endl;
        } else
                cout<<"Phone Number not found."<<endl;
        break;
case 4:
        cout<<"\nPhone Number: ";
        cin>>Old;
        pos=searchCustomer(customers, no_of_customers, Old);
        if (pos>=0)
                customers[pos].displayCustomerDetails();
        else
                cout<<"Customer not found."<<endl;
        break;
case 5:
        no_of_customers=0;
        cout<<"\nAll customers deleted."<<endl;
        break;
case 6:
                cout<<"Product Types:\n1) Phone\n2) Shoe"<<endl;
                cout<<"Your Choice: ";
                cin>>category;
                if (category<1 || category>2) {
                        cout<<"\nInvalid Choice."<<endl;
                        break;
                }
                if (category==1) {
                        if (no_of_phones<3) {
                                Phone p;
                                Product * p1=&p;
                        cout<<"\nProduct Name: ";
                        getline(cin, temp);
```

```
                    getline(cin, name);
                    cout<<"Price: ";
                    cin>>price;
                    cout<<"Quantity: ";
                    cin>>q;
                        p1->getDetails(p_id, price, name, q);
                        /*p.getDetails(p_id, price, name, q);*/
                        cout<<"Processor: ";
                        getline(cin, temp);
                        getline(cin, proc);
                        cout<<"Display: ";
                        getline(cin, disp);
                        cout<<"Battery Capacity (in mAh): ";
                        cin>>bat;
                        p.getDetails(proc, disp, bat);

                        phones[no_of_phones]=p;
        cout<<"\n"<<phones[no_of_phones].product_name<<" successfully
added."<<endl;
                        no_of_phones++;
                    } else
            cout<<"\nMaximum Phone Count Reached."<<endl;
                } else {
                    if (no_of_shoes<3) {
                        Shoe s;
                        Product * s1=&s;
                        cout<<"\nProduct Name: ";
                    getline(cin, temp);
                    getline(cin, name);
                    cout<<"Price: ";
                    cin>>price;
                    cout<<"Quantity: ";
                    cin>>q;
                        s1->getDetails(p_id, price, name, q);
                        cout<<"Brand Name: ";
                        getline(cin, temp);
                        getline(cin, brand);
                        cout<<"Shoe Type: ";
                        getline(cin, type);
                        cout<<"Size: ";
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_03/abstract_class$ g++ supermarket.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_03/abstract_class$ ./a.out

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 1

Phone Number: 7550099095
Name: Yuvaraj K
Customer Added.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 6
Product Types:
1) Phone
2) Shoe
Your Choice: 1
```

```
                                        cin>>size;
                                        s.getDetails(brand, type, size);
                                        shoes[no_of_shoes]=s;
        cout<<"\n"<<shoes[no_of_shoes].product_name<<" successfully
added."<<endl;
                                        no_of_shoes++;
                                } else
                cout<<"\nMaximum Shoe Count Reached."<<endl;
                                }
                                cout<<"Product ID: "<<p_id<<endl;
                        p_id++;
                        break;
                case 7:
                        cout<<"\nProduct ID: ";
                        cin>>id;
                        pos=searchPhone(phones, no_of_phones, id);
                        if (pos>=0) {
                                phones[pos].display();
                                } else {
                                        pos=searchShoe(shoes, no_of_shoes, id);
                                        if (pos>=0) {
                                                shoes[pos].display();
                                        } else
                                                cout<<"Product ID not found."<<endl;
                                }
                        break;
                        case 8:
                        cout<<"\nCustomer Phone Number: ";
                        cin>>cphone;
                cpos=searchCustomer(customers, no_of_customers, cphone);
                        if (cpos>=0) {
                                cout<<"Product ID: ";
                                cin>>pr_id;
                                ppos=searchPhone(phones, no_of_phones, pr_id);
                                if (ppos>=0) {
                                        cout<<"Quantity: ";
                                        cin>>qty;
                                        Phone p=phones[ppos];
                                        if (qty<=p.quantity) {
                                                p.quantity=qty;
```

```
Product Name: iPhone 15
Price: 800
Quantity: 100
Processor: A17 Bionic
Display: OLED
Battery Capacity (in mAh): 3700

iPhone 15 successfully added.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 6
Product Types:
1) Phone
2) Shoe
Your Choice: 2

Product Name: Air 7 Max
Price: 100
Quantity: 300
Brand Name: Nike
Shoe Type: Outdoor
Size: 10

Air 7 Max successfully added.
Product ID: 2
```

```
                                        phones[ppos].quantity-=qty;

                                        Bill b;
                                        b.createBill(b_id);
                                        b_id++;

                                        b.getCustomer(customers[cpos]);
                                        b.addPhone(p);
                                        cout<<"Bill Created.\nBill ID:
"<<b.bill_id<<endl;
                                        bills[no_of_bills]=b;
                                        no_of_bills++;
                                    } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                                } else {
                                        ppos=searchShoe(shoes, no_of_shoes, pr_id);
                                                if (ppos>=0) {
                                                    cout<<"Quantity: ";
                                                      cin>>qty;
                                                      Shoe p=shoes[ppos];
                                                      if (qty<=p.quantity) {
                                                            p.quantity=qty;
                                                            shoes[ppos].quantity-=qty;

                                                            Bill b;
                                                            b.createBill(b_id);
                                                            b_id++;
                                        b.getCustomer(customers[cpos]);
                                                            b.addShoe(p);
                        cout<<"Bill Created.\nBill ID: "<<b.bill_id<<endl;
                                                            bills[no_of_bills]=b;
                                                            no_of_bills++;
                                                      } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                                        } else
                                                cout<<"Product not found."<<endl;
                                }
                        } else
                                cout<<"Customer not found."<<endl;
                        break;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 8

Customer Phone Number: 7550099095
Product ID: 1
Quantity: 1
1 units of iPhone 15 has been added.
Bill Created.
Bill ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 9

Bill ID: 1
Product ID: 2
Quantity: 2
2 units of Air 7 Max has been added.
```

```
            case 9:
                cout<<"\nBill ID: ";
                cin>>id;
                pos=searchBill(bills, no_of_bills, id);
                if (pos>=0) {
                    cout<<"Product ID: ";
                    cin>>pr_id;
                    ppos=searchPhone(phones, no_of_phones, pr_id);
                    if (ppos>=0) {
                        cout<<"Quantity: ";
                        cin>>qty;
                        Phone p=phones[ppos];
                        if (qty<=p.quantity) {
                            p.quantity=qty;
                            phones[ppos].quantity-=qty;
                            bills[pos].addPhone(p);
                        } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                    } else {
                     ppos=searchShoe(shoes, no_of_shoes, pr_id);
                            if (ppos>=0) {
                                cout<<"Quantity: ";
                                cin>>qty;
                                Shoe p=shoes[ppos];
                                if (qty<=p.quantity) {
                                    p.quantity=qty;
                                    shoes[ppos].quantity-=qty;
                                    bills[pos].addShoe(p);
                                } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                             } else
                                    cout<<"Product ID not found."<<endl;
                    }
                } else
                    cout<<"Bill ID not found."<<endl;
                break;
            case 10:
                cout<<"\nBill ID: ";
                cin>>id;
                pos=searchBill(bills, no_of_bills, id);
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 10

Bill ID: 1

Customer Phone No: 7550099095
Customer Name: Yuvaraj K

No.     Product_Name    Unit_Price      Quantity        Total_Price

1       iPhone 15               800             1               800
1       Air 7 Max               100             2               200

Total Amount = 1000

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Create Bill
9. Update Existing Bill
10. Generate Bill Details
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_03/abstract_class$
```

```
                    if (pos>=0)
                        bills[pos].displayBill();
                    else
                        cout<<"Bill ID not found."<<endl;
                    break;
                default:
                    cout<<"Invalid Choice."<<endl;
                    break;
            }
        }
        return 0;
}
```

RESULT

The various types of inheritance, virtual functions, and abstract classes were all successfully implemented in the supermarket management system application.

Exp. No: 4
Date: 05/03/2024

IMPLEMENTATION OF PROGRAMS USING FUNCTION TEMPLATES AND CLASS TEMPLATES

AIM

To implement logical ideas of function templates and class templates in the supermarket management system application.

ALGORITHM

Introduce a template T for Product class. Price can be stored as double or float or any other similar data type, and so the template variable T will be for the price.

Introduce a template U for the display function. If a string is to be displayed, name will be returned; if an integer is to be displayed, quantity will be returned, and if a value of type T (which shouldn't be int or string) is to be displayed, price will be returned. So, one display function is enough to return any of the three values stored in Product class.

```
template T
class Product {
        string name, int quantity, T price;
        getDetails();

        template U
        U display() {
                if U is string, return name
                if U is int, return quantity
                if U is T, return price
        }
}
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_04$ g++ binarySearch.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_04$ ./a.out
Searching 2 in Integer Array
2 is found at position 1.
Searching e in Float Array
e is found at position 4.
Searching 5.7 in Character Array
5.7 is found at position 2.
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_04$
```

SOURCE CODE

Implementation of Binary Search using Function Template

```
#include <iostream>
using namespace std;
template <typename T>
int binarySearch(T a[], T key, int low, int high) {
	if (low<=high) {
		int mid=(low+high)/2;
		if (key==a[mid])
			return mid;
		else if (key>a[mid])
			return binarySearch(a, key, mid+1, high);
		else if (key<a[mid])
			return binarySearch(a, key, low, mid-1);
	}
	return -1;
}
int main() {
	int pos;
	int a[5]={1,2,3,4,5}, k1=2;
	char b[5]={'a','b','c','d','e'}, k2='e';
	float c[5]={1.1, 3.4, 5.7, 8.6, 9.8}, k3=5.7;
	cout<<"Searching "<<k1<<" in Integer Array"<<endl;
	if (pos=binarySearch(a, k1, 0, 4))
		cout<<k1<<" is found at position "<<pos<<"."<<endl;
	else
		cout<<k1<<" is not found."<<endl;
	cout<<"Searching "<<k2<<" in Float Array"<<endl;
	if (pos=binarySearch(b, k2, 0, 4))
		cout<<k2<<" is found at position "<<pos<<"."<<endl;
	else
		cout<<k2<<" is not found."<<endl;
	cout<<"Searching "<<k3<<" in Character Array"<<endl;
	if (pos=binarySearch(c, k3, 0, 4))
		cout<<k3<<" is found at position "<<pos<<"."<<endl;
	else
		cout<<k3<<" is not found."<<endl;
}
```

Implementation in Supermarket Application

```
#include <iostream>
#include <string>
using namespace std;

template <typename T>
class Product {
	private:
		string name, temp;
		int quantity;
		T price;
	public:
		void getProductDetails() {
			cout<<"\nName: ";
			getline(cin, temp);
			getline(cin, name);
			cout<<"Price: ";
			cin>>price;
			cout<<"Quantity: ";
			cin>>quantity;
		}
		template <typename U>
		U display() {
			if constexpr (is_same<U, int>::value)
				return quantity;
			else if constexpr (is_same<U, string>::value)
				return name;
			else if constexpr (is_same<U, T>::value)
				return price;
		}
};
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_04$ g++ supermarket.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_04$ ./a.out
Getting Product Details
No. of Products: 3

Name: iPhone 15
Price: 800
Quantity: 100

Name: Nike Air 7
Price: 100
Quantity: 200

Name: Men's Shirt
Price: 20
Quantity: 200

Product Name    Price   Available Quantity
iPhone 15       800     100
Nike Air 7      100     200
Men's Shirt     20      200
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_04$ |
```

```
int main() {
	int no;
	cout<<"Getting Product Details"<<endl;
	cout<<"No. of Products: ";
	cin>>no;
	Product <double> * products=new Product<double>[no];
	for (int i=0; i<no; i++) {
		Product <double> p;
		p.getProductDetails();
		products[i]=p;
	}
	cout<<"\nProduct Name\tPrice\tAvailable Quantity"<<endl;
	for (int i=0; i<no; i++) {
		cout<<products[i].display<string>()<<"\t";
		cout<<products[i].display<double>()<<"\t";
		cout<<products[i].display<int>()<<endl;
	}
	return 0;
}
```

RESULT

Function and class templates were successfully implemented for the supermarket management system application.

Exp. No: 5
Date: 12/03/2024

IMPLEMENTATION OF A SUPERMARKET MANAGEMENT SYSTEM USING OBJECT ORIENTED PROGRAMMING CONCEPTS

AIM

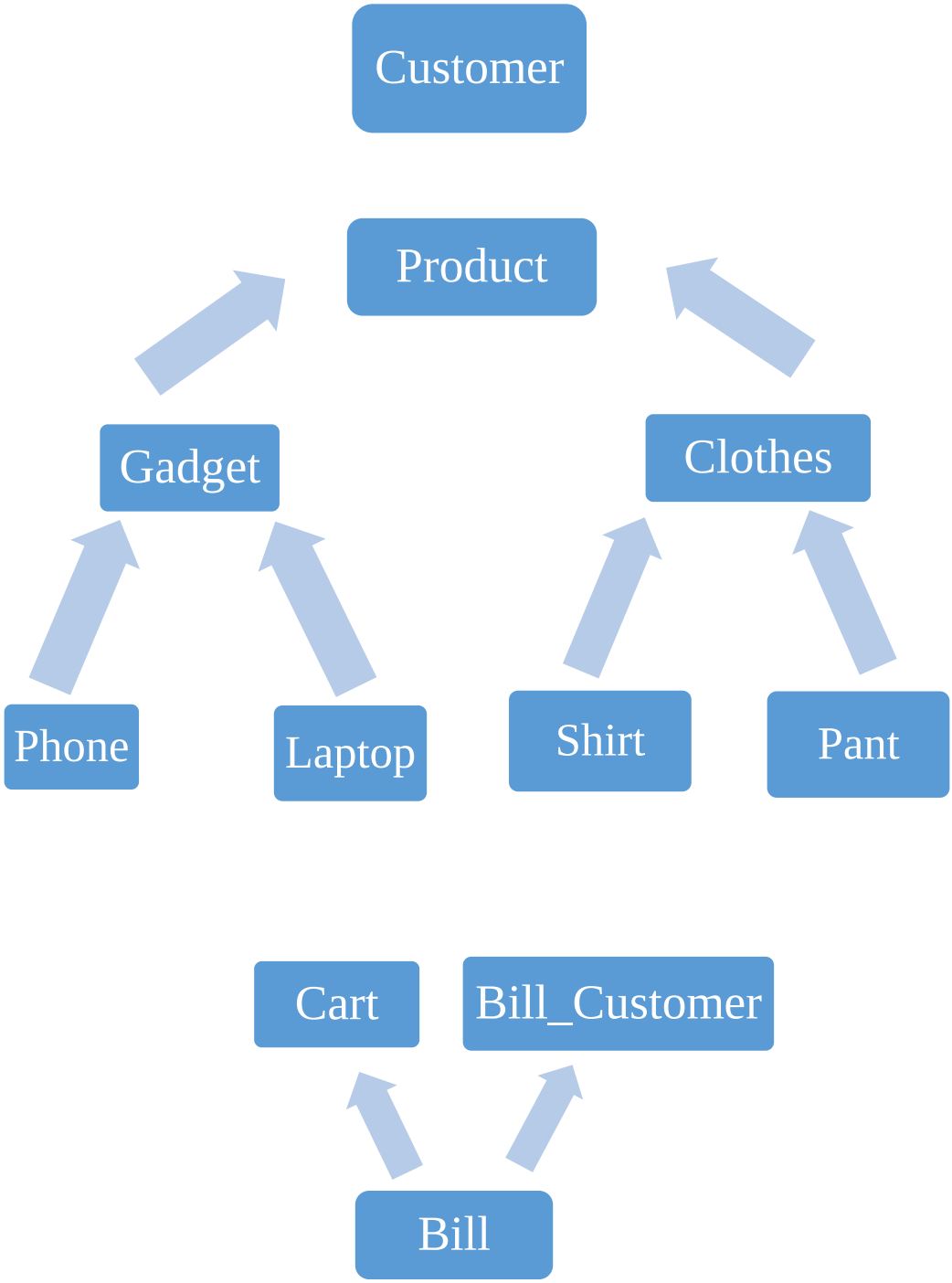
To implement a supermarket management application system using various object oriented programming concepts such as classes, objects, constructors, function overloading, operating overloading, inheritance, virtual functions, and templates.

DESCRIPTION

A traditional supermarket environment consists of various entities, such as customers, products, and bills. It is important to keep track of the various attributes belonging to each of these entities, such as the customer name, the product name and price, the list of products and quantities in the bill, and many, many more. There can also be human errors made by employees while registering a customer or creating a bill, and there needs to be simple and effective mechanisms to handle those.

Using the object oriented programming concepts learned up to now, it is possible to create a comprehensive, exhaustive management system for a supermarket-based application.

Classes can be created for all the entities mentioned above, with appropriate data members covering the necessary attributes required. Member functions that work on these data members can be created with the necessary utilities to bring out fruitful results. Concepts of function overloading, constructor overloading, inheritance and templates can be effectively used to simplify the code structure and add a more detail-oriented and descriptive view. Operator overloading can be utilised in order to bring out additional, unique features tailored to the application. Pure virtual functions and abstract classes can be implemented in order to make the code universal and allow any developer or programmer to work on it with a clear idea of what is happening and how to proceed.


Customer
Product
Gadget
Clothes
Phone
Laptop
Shirt
Pant
Cart
Bill_Customer
Bill

Thus, a well-detailed and thorough application covering nearly all the aspects of a supermarket management system can be implemented in a systematic and and structured manner.

Some of the many functionalities that can be accomplished for this application are:

- Registration of a new customer
- Updating the details of an existing customer
    - Change customer's name
    - Update customer's phone number
- Adding a new product
- Updating the available stock of the product
    - Receiving new imports of the product and thus increasing the available stock
    - Exporting a set quantity of products and thus decreasing the available stock
- Updating the price of the product
- Creating bills that link the customer and a set of products
- Adding more and more items to existing bills
- Displaying customer, product, and bill details as requested
- Combining two products which is actually one product that was accidentally created twice.
- Combining multiple bills belonging to the same customer that were either accidentally created separately or were requested to be combined by the customer who initially wanted them to be separate.

```
warspinix@Yuvaraj-Laptop:~/c++/supermarket$ g++ supermarket.cpp
warspinix@Yuvaraj-Laptop:~/c++/supermarket$ ./a.out

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 1

Phone Number: 8056017760
Name: Yuvaraj
Customer Added.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 4
```

SOURCE CODE

supermarket.h

```
#include <iostream>
#include <string>
using namespace std;

class Customer {
	public:
		string phone_no;
		string name;
		string temp;
		int isValid(string pno) {
			if (pno.length()!=10) {
				cout<<"Invalid Phone Number."<<endl;
				return 0;
			}
			return 1;
		}
		int getCustomerDetails() {
			getline(cin, temp);
			cout<<"\nPhone Number: ";
			getline(cin, phone_no);
			if (isValid(phone_no)) {
				cout<<"Name: ";
				getline(cin, name);
				return 1;
			}
			return 0;
		}
		int updatePhoneNo (string newno) {
			if (isValid(newno)) {
				phone_no=newno;
				return 1;
			}
			return 0;
		}
```

```
Phone Number: 8056017760

Phone No: 8056017760
Customer Name: Yuvaraj

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 2

Enter Old Phone Number: 8056017760
Enter New Phone Number: 7550099095
Update Successful.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 3
```

```
 void updateName (string newname) {
                name=newname;
        }
        void displayCustomerDetails() {
                cout<<"\nPhone No: "<<phone_no<<endl;
                cout<<"Customer Name: "<<name<<endl;
        }
};

class Product {
    public:
        int product_id, quantity;
        float price;
        string product_name;
        void getProductDetails (int id, float p, string name, int q) {
                product_id=id;
                product_name=name;
                price=p;
                quantity=q;
        }
        void update (string newname) {
                cout<<"\nOld Name: "<<product_name<<endl;
                product_name=newname;
                cout<<"Product Name successfully updated."<<endl;
                cout<<"New Name: "<<product_name<<endl;
        }
        void update (float p) {
                cout<<"\nOld Price = "<<price<<endl;
                price=p;
                cout<<"Price of "<<product_name<<" successfully
updated."<<endl;
                cout<<"New Price = "<<price<<endl;
        }
        void update (int q) {
                cout<<"\nOld Quantity = "<<quantity<<endl;
                quantity+=q;
                cout<<"Quantity of "<<product_name<<" successfully
updated."<<endl;
                cout<<"New Quantity = "<<quantity<<endl;
        }
```

```
Phone Number: 7550099095
Enter New Name: Yuvaraj K
Update Successful.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 4

Phone Number: 8056017760
Customer not found.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 4
```

```
void displayProductDetails() {
                cout<<"\nProduct Name: "<<product_name<<endl;
                cout<<"Price: "<<price<<endl;
                cout<<"Available Quantity: "<<quantity<<endl;
        }
};

class Gadget: public Product {
    public:
        string brand, display;
        int battery;
        void getGadgetDetails(string b, string d, int bat) {
                brand=b;
                display=d;
                battery=bat;
        }
        void displayGadgetDetails() {
                cout<<"Brand: "<<brand<<endl;
                cout<<"Display Technology: "<<display<<endl;
                cout<<"Battery Capacity: "<<battery<<endl;
        }
};

class Phone: public Gadget {
    public:
        int cameras, unlock;
        string connection;
        void getPhoneDetails(int c, int u, string con) {
                cameras=c;
                unlock=u;
                connection=con;
        }
        Phone operator + (Phone const &p) {
                Phone result;
                result.product_id=product_id;
                result.quantity=quantity+p.quantity;
                result.price=price;
                result.product_name=product_name;
                result.brand=brand;
                result.display=display;
```

```
Phone Number: 8056017760
Customer not found.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 4

Phone Number: 7550099095

Phone No: 7550099095
Customer Name: Yuvaraj K

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 6
```

```
                result.battery=battery;
                result.cameras=cameras;
                result.unlock=unlock;
                result.connection=connection;
                return result;
            }
            void displayPhoneDetails() {
                cout<<"No. of Cameras: "<<cameras<<endl;
                cout<<"No. of Unlock Methods: "<<unlock<<endl;
                cout<<"Connection: "<<connection<<endl;
            }
};
class Laptop: public Gadget {
    public:
            string fans, trackpad, keyboard;
            void getLaptopDetails(string f, string t, string k) {
                fans=f;
                trackpad=t;
                keyboard=k;
            }
            Laptop operator + (Laptop const &p) {
                Laptop result;
                result.product_id=product_id;
                result.quantity=quantity+p.quantity;
                result.price=price;
                result.product_name=product_name;
                result.brand=brand;
                result.display=display;
                result.battery=battery;
                result.fans=fans;
                result.trackpad=trackpad;
                result.keyboard=keyboard;
                return result;
            }
            void displayLaptopDetails() {
                cout<<"Fans: "<<fans<<endl;
                cout<<"Trackpad: "<<trackpad<<endl;
                cout<<"Keyboard: "<<keyboard<<endl;
            }
};
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1

Product Name: iPhone 15 Pro
Price: 1000
Quantity: 200
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Brand Name: Apple
Display Technology: OLED
Battery Capacity: 4000
No. of Cameras: 4
No. of Unlock Methods: 2
Network Connection: 5G

iPhone 15 Pro successfully added.
Product ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 6
```

```
class Clothes: public Product {
	public:
		string material, type;
		float size;
		void getClothDetails (string m, string t, float s) {
			material=m;
			type=t;
			size=s;
		}
		void displayClothDetails () {
			cout<<"Material: "<<material<<endl;
			cout<<"Type: "<<type<<endl;
			cout<<"Cloth Size: "<<size<<endl;
		}
};
class Shirt: public Clothes {
	public:
		string buttons, length;
		void getShirtDetails(string b, string l) {
			buttons=b;
			length=l;
		}
		Shirt operator + (Shirt const &p) {
			Shirt result;
			result.product_id=product_id;
			result.quantity=quantity+p.quantity;
			result.price=price;
			result.product_name=product_name;
			result.material=material;
			result.type=type;
			result.size=size;
			result.buttons=buttons;
			result.length=length;
			return result;
		}
		void displayShirtDetails() {
			cout<<"Button Type: "<<buttons<<endl;
			cout<<"Arm-Length: "<<length<<endl;
		}
};
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1

Product Name: Dell G15
Price: 800
Quantity: 100
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 2
Brand Name: Dell
Display Technology: LCD
Battery Capacity: 8000
Fans: 2
Trackpad: Multi-Touch
Keyboard: Chiclet-Style

Dell G15 successfully added.
Product ID: 2

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 6
```

```
class Pant: public Clothes {
	public:
		int pockets;
		string button;
		void getPantDetails(int p, string b) {
			pockets=p;
			button=b;
		}
		Pant operator + (Pant const &p) {
			Pant result;
			result.product_id=product_id;
			result.quantity=quantity+p.quantity;
			result.price=price;
			result.product_name=product_name;
			result.material=material;
			result.type=type;
			result.size=size;
			result.pockets=pockets;
			result.button=button;
			return result;
		}
		void displayPantDetails() {
			cout<<"No. of Pockets: "<<pockets<<endl;
			cout<<"Pant Button Type: "<<button<<endl;
		}

};

class Cart {
	public:
		Phone * phones=new Phone[3];
		Laptop * laptops=new Laptop[3];
		Shirt * shirts=new Shirt[3];
		Pant* pants=new Pant[3];
		int no_of_phones, no_of_laptops;
		int no_of_shirts, no_of_pants;
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 2

Product Name: Liverpool 24/25
Price: 30
Quantity: 250
Type of Cloth:
        1. Shirt
        2. Pant
Your Choice: 1
Material: Cotton
Type: Jersey
Size: 39
Button Type: None
Arm-Length: Half-Sleeved

Liverpool 24/25 successfully added.
Product ID: 3

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 6
```

```
template <typename T>
int check (T p) {
	int total;
	T * products;
	if constexpr (is_same<T, Phone>::value) {
		total=no_of_phones;
		products=phones;
	} else if constexpr (is_same<T, Laptop>::value) {
		total=no_of_laptops;
		products=laptops;
	} else if constexpr (is_same<T, Shirt>::value) {
		total=no_of_shirts;
		products=shirts;
	} else if constexpr (is_same<T, Pant>::value) {
		total=no_of_pants;
		products=pants;
	}
	for (int i=0; i<total; i++)
		if (products[i].product_id==p.product_id)
			return i;
	return -1;
}
template <typename T>
void add (T p) {
	int pos=check(p);
	if constexpr (is_same<T, Phone>::value) {
		if (pos>=0) {
			phones[pos].quantity+=p.quantity;
			cout<<p.product_name<<" already exists.
"<<p.quantity<<" units of "<<p.product_name<<" has been added."<<endl;
		} else {
			phones[no_of_phones]=p;
			cout<<p.quantity<<" units of "<<p.product_name<<"
has been added."<<endl;
			no_of_phones++;
		}
	} else if constexpr (is_same<T, Laptop>::value) {
		if (pos>=0) {
			laptops[pos].quantity+=p.quantity;
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 2

Product Name: Adidas X3
Price: 25
Quantity: 50
Type of Cloth:
        1. Shirt
        2. Pant
Your Choice: 2
Material: Linen
Type: Jersey
Size: 32
No. of Pockets: 2
Pant Button Type: None

Adidas X3 successfully added.
Product ID: 4

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 6
```

```
cout<<p.product_name<<" already exists. "<<p.quantity<<" units of
"<<p.product_name<<" has been added."<<endl;
                        } else {
                                laptops[no_of_laptops]=p;
                                cout<<p.quantity<<" units of "<<p.product_name<<"
has been added."<<endl;
                                no_of_laptops++;
                        }

                } else if constexpr (is_same<T, Shirt>::value) {
                        if (pos>=0) {
                                shirts[pos].quantity+=p.quantity;
                                cout<<p.product_name<<" already exists.
"<<p.quantity<<" units of "<<p.product_name<<" has been added."<<endl;
                        } else {
                                shirts[no_of_shirts]=p;
                                cout<<p.quantity<<" units of "<<p.product_name<<"
has been added."<<endl;
                                no_of_shirts++;
                        }

                } else if constexpr (is_same<T, Pant>::value) {
                        if (pos>=0) {
                                pants[pos].quantity+=p.quantity;
                                cout<<p.product_name<<" already exists.
"<<p.quantity<<" units of "<<p.product_name<<" has been added."<<endl;
                        } else {
                                pants[no_of_pants]=p;
                                cout<<p.quantity<<" units of "<<p.product_name<<"
has been added."<<endl;
                                no_of_pants++;
                        }
                }
        }
};
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1

Product Name: iphone 15 pro
Price: 1000
Quantity: 25
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Brand Name: Apple
Display Technology: OLED
Battery Capacity: 4000
No. of Cameras: 4
No. of Unlock Methods: 2
Network Connection: 5G

iphone 15 pro successfully added.
Product ID: 5

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 7
```

```
class Bill: public Cart {
	public:
		int bill_id;
		Customer customer;
		float total;
		void createBill (int id) {
			bill_id=id;
			no_of_phones=0;
			no_of_laptops=0;
			no_of_shirts=0;
			no_of_pants=0;
		}
		void getCustomer(Customer c) {
			customer=c;
		}
		Bill operator + (Bill const &b) {
			Bill result;
			for (int i=0; i<no_of_phones; i++) {
				int c=1;
				for (int j=0; j<no_of_phones; j++) {
					if (phones[j].product_id==b.phones[i].product_id) {
						phones[j].quantity+=b.phones[i].quantity;
						c=0;
						break;
					}
				}
				if (c) {
					phones[no_of_phones]=b.phones[i];
					no_of_phones++;
				}
			}
			for (int i=0; i<no_of_laptops; i++) {
				int c=1;
				for (int j=0; j<no_of_laptops; j++) {
					if (laptops[j].product_id==b.laptops[i].product_id) {
						laptops[j].quantity+=b.laptops[i].quantity;
						c=0;
						break;
					}
				}
```

```
Your Choice: 7
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Product ID: 1

Product Name: iPhone 15 Pro
Price: 1000
Available Quantity: 200
Brand: Apple
Display Technology: OLED
Battery Capacity: 4000
No. of Cameras: 4
No. of Unlock Methods: 2
Connection: 5G

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 7
```

```
        if (c) {
            laptops[no_of_laptops]=b.laptops[i];
            no_of_laptops++;
        }
    }
    for (int i=0; i<no_of_shirts; i++) {
        int c=1;
        for (int j=0; j<no_of_shirts; j++) {
            if (shirts[j].product_id==b.shirts[i].product_id) {
                shirts[j].quantity+=b.shirts[i].quantity;
                c=0;
                break;
            }
        }
        if (c) {
            shirts[no_of_shirts]=b.shirts[i];
            no_of_shirts++;
        }
    }
    for (int i=0; i<no_of_pants; i++) {
        int c=1;
        for (int j=0; j<no_of_pants; j++) {
            if (pants[j].product_id==b.pants[i].product_id) {
                pants[j].quantity+=b.pants[i].quantity;
                c=0;
                break;
            }
        }
        if (c) {
            pants[no_of_pants]=b.pants[i];
            no_of_pants++;
        }
    }
    result.bill_id=bill_id;
    result.customer=customer;
    result.phones=phones;
    result.no_of_phones=no_of_phones;
    result.laptops=laptops;
    result.no_of_laptops=no_of_laptops;
    result.shirts=shirts;
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Product ID: 5

Product Name: iphone 15 pro
Price: 1000
Available Quantity: 25
Brand: Apple
Display Technology: OLED
Battery Capacity: 4000
No. of Cameras: 4
No. of Unlock Methods: 2
Connection: 5G

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 8
```

```
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
ID of Product 1: 1
ID of Product 2: 5
Products Successfully Combined.
Product ID: 1
```

```
		result.no_of_shirts=no_of_shirts;
		result.pants=pants;
		result.no_of_pants=no_of_pants;
		return result;
	}
	void computeTotal() {
		total=0;
		for (int i=0; i<no_of_phones; i++)
			total+=(phones[i].price*phones[i].quantity);
		for (int i=0; i<no_of_laptops; i++)
			total+=(laptops[i].price*laptops[i].quantity);
		for (int i=0; i<no_of_shirts; i++)
			total+=(shirts[i].price*shirts[i].quantity);
		for (int i=0; i<no_of_pants; i++)
			total+=(pants[i].price*pants[i].quantity);
	}
	void displayBill() {
		string name;
		computeTotal();
		cout<<"\nCustomer Phone No: "<<customer.phone_no<<endl;
		cout<<"Customer Name: "<<customer.name<<endl;
		cout<<"\n|No.\t|Product Name\t|Unit Price\t|Quantity\t|Total
Price\t|"<<endl;
		int i, j, k, l;
		for (i=0; i<no_of_phones; i++) {
			cout<<"|"<<i+1<<"\t|";
			name=phones[i].product_name;
			if (name.length()<7)
				cout<<name<<"\t\t|";
			else
				cout<<name.substr(0, 11)<<"...\t|";
			cout<<phones[i].price<<"\t\t|";
			cout<<phones[i].quantity<<"\t\t|";
			cout<<phones[i].price*phones[i].quantity<<"\t\t|"<<endl;
		}
		for (j=0; j<no_of_laptops; j++) {
			cout<<"|"<<i+j+1<<"\t|";
			name=laptops[j].product_name;
			if (name.length()<7)
				cout<<name<<"\t\t|";
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 7
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Product ID: 5
Product ID not found.

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 9
```

```
                else
                        cout<<name.substr(0, 11)<<"...\t|";
                cout<<laptops[j].price<<"\t\t|";
                cout<<laptops[j].quantity<<"\t\t|";
                cout<<laptops[j].price*laptops[j].quantity<<"\t\t|"<<endl;
        }
        for (k=0; k<no_of_shirts; k++) {
                cout<<"|"<<i+j+k+1<<"\t|";
                name=shirts[k].product_name;
                if (name.length()<7)
                        cout<<name<<"\t\t|";
                else
                        cout<<name.substr(0, 11)<<"...\t|";
                cout<<shirts[k].price<<"\t\t|";
                cout<<shirts[k].quantity<<"\t\t|";
                cout<<shirts[k].price*shirts[k].quantity<<"\t\t|"<<endl;
        }
        for (l=0; l<no_of_pants; l++) {
                cout<<"|"<<i+j+k+l+1<<"\t|";
                name=pants[l].product_name;
                if (name.length()<7)
                        cout<<name<<"\t\t|";
                else
                        cout<<name.substr(0, 11)<<"...\t|";
                cout<<pants[l].price<<"\t\t|";
                cout<<pants[l].quantity<<"\t\t|";
                cout<<pants[l].price*pants[l].quantity<<"\t\t|"<<endl;
        }
        cout<<"\nTotal Amount = "<<total<<endl;
    }
};

int searchCustomer (Customer * cs, int total, string p_no) {
    for (int i=0; i<total; i++) {
        if (cs[i].phone_no==p_no)
            return i;
    }
    return -1;
}
```

```
Customer Phone Number: 7550099095
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Product ID: 1
Quantity: 2
2 units of iPhone 15 Pro has been added.
Bill Created
Bill ID: 1

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 7
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 1
Product ID: 1
```

```
template <typename T>
int search (T * products, int total, int p_id) {
	for (int i=0; i<total; i++)
		if (products[i].product_id==p_id)
			return i;
	return -1;
}

int searchBill (Bill * bs, int total, int b_id) {
	for (int i=0; i<total; i++)
		if (bs[i].bill_id==b_id)
			return i;
	return -1;
}
```

```
Product Name: iPhone 15 Pro
Price: 1000
Available Quantity: 223
Brand: Apple
Display Technology: OLED
Battery Capacity: 4000
No. of Cameras: 4
No. of Unlock Methods: 2
Connection: 5G

0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 10

Bill ID: 1
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 1
Type of Device:
        1. Phone
        2. Laptop
Your Choice: 2
Product ID: 2
Quantity: 1
1 units of Dell G15 has been added.
```

supermarket.cpp

```
#include <iostream>
#include <string>
#include <cctype>
#include "supermarket.h"
using namespace std;

int main() {

	// General Variabless
	int choice, pos, cpos, ppos, category;

	// Customer Variables
	string Old, New;
	Customer * customers=new Customer[3];
	int no_of_customers=0;

	// Product Variables
	int p_id=1;
	string name, temp;
	float price;
	int id, q;
	Product * products=new Product[9];
	int no_of_products=0;

	// Gadget Variables
	string brand, disp;
	int gb;

	// Phone Variables
	int cam, un;
	string con;
	Phone * phones=new Phone[9];
	int no_of_phones=0;

	// Laptop Variables
	string fan, track, kb;
	Laptop * laptops=new Laptop[9];
	int no_of_laptops=0;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 10

Bill ID: 1
Product Category:
        1. Gadget
        2. Cloth
Your Choice: 2
Type of Cloth:
        1. Shirt
        2. Pant
Your Choice: 1
Product ID: 3
Quantity: 10
10 units of Liverpool 24/25 has been added.
```

```
// Cloth Variables
string mat, type;
float size;

// Shirt Variables
string sb, len;
Shirt * shirts=new Shirt[9];
int no_of_shirts=0;

// Pant Variables
int pocket;
string pb;
Pant * pants=new Pant[9];
int no_of_pants=0;

// Bill Variables
int b_id=1;
string cphone;
int pr_id, qty;
Bill * bills=new Bill[3];
int no_of_bills=0;

// Variables for Operator Overloading
int o1, o2, o1pos, o2pos;
int np1, np2, t1, t2, t3, t4;

while (1) {
	cout<<"\n0. Exit\n1. Add Customer\n2. Update Customer Phone
Number\n3. Update Customer Name\n4. Display Customer Details\n5. Delete
All Customers\n6. Add Product\n7. Display Product Details\n8. Combine
Products\n9. Create Bill\n10. Add to Existing Bill\n11. Combine Bill\n12.
Generate Bill Details\n";
	cout<<"Your Choice: ";
	cin>>choice;
	if (choice==0)
		break;
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 12

Bill ID: 1

Customer Phone No: 7550099095
Customer Name: Yuvaraj K

|No.     |Product Name   |Unit Price     |Quantity       |Total Price    |
|1       |iPhone 15 P... |1000           |2              |2000           |
|2       |Dell G15...    |800            |1              |800            |
|3       |Liverpool 2... |30             |10             |300            |

Total Amount = 3100
```

```
0. Exit
1. Add Customer
2. Update Customer Phone Number
3. Update Customer Name
4. Display Customer Details
5. Delete All Customers
6. Add Product
7. Display Product Details
8. Combine Products
9. Create Bill
10. Add to Existing Bill
11. Combine Bill
12. Generate Bill Details
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++/supermarket$
```

```
switch (choice) {
        case 1:
            if (no_of_customers<3) {
                Customer c;
                if (c.getCustomerDetails()) {
                    cout<<"Customer Added."<<endl;
                    customers[no_of_customers]=c;
                    no_of_customers++;
                }
            } else
                cout<<"Maximum Customer Count Reached."<<endl;
            break;
        case 2:
            cout<<"\nEnter Old Phone Number: ";
            cin>>Old;
            pos=searchCustomer(customers, no_of_customers, Old);
            if (pos>=0) {
                cout<<"Enter New Phone Number: ";
                cin>>New;
                if (customers[pos].updatePhoneNo(New))
                    cout<<"Update Successful."<<endl;
                else
                    cout<<"Update Failed."<<endl;
            } else
                cout<<"Phone Number not found."<<endl;
            break;
        case 3:
            cout<<"\nPhone Number: ";
            cin>>Old;
            pos=searchCustomer(customers, no_of_customers, Old);
            if (pos>=0) {
                getline(cin, temp);
                cout<<"Enter New Name: ";
                getline(cin, New);
                customers[pos].updateName(New);
                cout<<"Update Successful."<<endl;
            } else
                cout<<"Phone Number not found."<<endl;
            break;
```

```
case 4:
                cout<<"\nPhone Number: ";
                cin>>Old;
                pos=searchCustomer(customers, no_of_customers, Old);
                if (pos>=0)
                        customers[pos].displayCustomerDetails();
                else
                        cout<<"Customer not found."<<endl;
                break;
        case 5:
                no_of_customers=0;
                cout<<"\nAll customers deleted."<<endl;
                break;
        case 6:
                cout<<"Product Category:\n\t1. Gadget\n\t2. Cloth"<<endl;
                cout<<"Your Choice: ";
                cin>>category;
                if (category<1 || category>2)
                        break;
                if (no_of_products<10) {
                        cout<<"\nProduct Name: ";
                        getline(cin, temp);
                        getline(cin, name);
                        cout<<"Price: ";
                        cin>>price;
                        cout<<"Quantity: ";
                        cin>>q;
                        if (category==1) {
                                cout<<"Type of Device:\n\t1. Phone\n\t2.
Laptop"<<endl;
                                cout<<"Your Choice: ";
                                cin>>category;
                                if (category<1 || category>2)
                                        break;
                                cout<<"Brand Name: ";
                                getline(cin, temp);
                                getline(cin, brand);
                                cout<<"Display Technology: ";
                                getline(cin, disp);
                                cout<<"Battery Capacity: ";
```

```
cin>>gb;
if  (category==1) {
        Phone p;
        p.getProductDetails(p_id, price, name, q);
        p.getGadgetDetails(brand, disp, gb);
        cout<<"No. of Cameras: ";
        cin>>cam;
        cout<<"No. of Unlock Methods: ";
        cin>>un;
        cout<<"Network Connection: ";
        getline(cin, temp);
        getline(cin, con);
        p.getPhoneDetails(cam, un, con);
        phones[no_of_phones]=p;

cout<<"\n"<<phones[no_of_phones].product_name<<" successfully
added."<<endl;
        no_of_phones++;
} else {
        Laptop p;
        p.getProductDetails(p_id, price, name, q);
        p.getGadgetDetails(brand, disp, gb);
        cout<<"Fans: ";
        getline(cin, temp);
        getline(cin, fan);
        cout<<"Trackpad: ";
        getline(cin, track);
        cout<<"Keyboard: ";
        getline(cin, kb);
        p.getLaptopDetails(fan, track, kb);
        laptops[no_of_laptops]=p;

cout<<"\n"<<laptops[no_of_laptops].product_name<<" successfully
added."<<endl;
        no_of_laptops++;
}
```

```
                    else {
                        cout<<"Type of Cloth:\n\t1. Shirt\n\t2.
Pant"<<endl;
                        cout<<"Your Choice: ";
                        cin>>category;
                        if (category<1 || category>2)
                            break;
                        cout<<"Material: ";
                        getline(cin, temp);
                        getline(cin, mat);
                        cout<<"Type: ";
                        getline(cin, type);
                        cout<<"Size: ";
                        cin>>size;
                        if  (category==1) {
                            Shirt p;
                            p.getProductDetails(p_id, price, name, q);
                            p.getClothDetails(mat, type, size);
                            cout<<"Button Type: ";
                            getline(cin, temp);
                            getline(cin, sb);
                            cout<<"Arm-Length: ";
                            getline(cin, len);
                            p.getShirtDetails(sb, len);
                            shirts[no_of_shirts]=p;

cout<<"\n"<<shirts[no_of_shirts].product_name<<" successfully
added."<<endl;
                            no_of_shirts++;
                        }
```

```
                    } else {
                        Pant p;
                        p.getProductDetails(p_id, price, name, q);
                        p.getClothDetails(mat, type, size);
                        cout<<"No. of Pockets: ";
                        cin>>pocket;
                        cout<<"Pant Button Type: ";
                        getline(cin, temp);
                        getline(cin, pb);
                        p.getPantDetails(pocket, pb);
                        pants[no_of_pants]=p;

cout<<"\n"<<pants[no_of_pants].product_name<<" successfully
added."<<endl;
                        no_of_pants++;
                    }
                }
                no_of_products++;
                cout<<"Product ID: "<<p_id<<endl;
                p_id++;
            } else {
                cout<<"Maximum Product Count Reached."<<endl;
            }
            break;
        case 7:
            cout<<"Product Category:\n\t1. Gadget\n\t2. Cloth"<<endl;
            cout<<"Your Choice: ";
            cin>>category;
            if (category<1 || category>2)
                break;
            if (category==1) {
              cout<<"Type of Device:\n\t1. Phone\n\t2. Laptop"<<endl;
                cout<<"Your Choice: ";
                cin>>category;
                if (category<1 || category>2)
                    break;
```

```
        if (category==1) {
            cout<<"Product ID: ";
            cin>>id;
            pos=search(phones, no_of_phones, id);
            if (pos>=0) {
                phones[pos].displayProductDetails();
                phones[pos].displayGadgetDetails();
                phones[pos].displayPhoneDetails();
            } else
                cout<<"Product ID not found."<<endl;
        } else {
            cout<<"Product ID: ";
            cin>>id;
            pos=search(laptops, no_of_laptops, id);
            if (pos>=0) {
                laptops[pos].displayProductDetails();
                laptops[pos].displayGadgetDetails();
                laptops[pos].displayLaptopDetails();
            } else
                cout<<"Product ID not found."<<endl;
        }
    } else {
        cout<<"Type of Cloth:\n\t1. Shirt\n\t2. Pant"<<endl;
        cout<<"Your Choice: ";
        cin>>category;
        if (category<1 || category>2)
            break;
        if (category==1) {
            cout<<"Product ID: ";
            cin>>id;
            pos=search(shirts, no_of_shirts, id);
            if (pos>=0) {
                shirts[pos].displayProductDetails();
                shirts[pos].displayClothDetails();
                shirts[pos].displayShirtDetails();
            } else
                cout<<"Product ID not found."<<endl;
        }
```

```
                else {
                        cout<<"Product ID: ";
                        cin>>id;
                        pos=search(pants, no_of_pants, id);
                        if (pos>=0) {
                                pants[pos].displayProductDetails();
                                pants[pos].displayClothDetails();
                                pants[pos].displayPantDetails();
                        } else
                                cout<<"Product ID not found."<<endl;
                }
        }
        break;
case 8:
        cout<<"Product Category:\n\t1. Gadget\n\t2. Cloth"<<endl;
        cout<<"Your Choice: ";
        cin>>category;
        if (category<1 || category>2)
                break;
        if (category==1) {
          cout<<"Type of Device:\n\t1. Phone\n\t2. Laptop"<<endl;
                cout<<"Your Choice: ";
                cin>>category;
                if (category<1 || category>2)
                        break;
                if (category==1) {
                        cout<<"ID of Product 1: ";
                        cin>>o1;
                        o1pos=search(phones, no_of_phones, o1);
                        if (o1pos>=0) {
                                cout<<"ID of Product 2: ";
                                cin>>o2;
                                o2pos=search(phones, no_of_phones, o2);
```

```
                    if (o2pos>=0) {
                         if (o1==o2)
                        cout<<"The IDs are the same."<<endl;
                         else {
                        string n1=phones[o1pos].product_name;
                        string n2=phones[o2pos].product_name;
                              int l1=n1.length();
                              int l2=n2.length();
                             if (l1==l2) {
                                    int c=1;
                                    for (int i=0; i<l1; i++) {
                                if (toupper(n1[i]) != toupper(n2[i])) {
                                                c=0;
                                                break;
                                          }
                                    }
                                    if (c) {
                                          if (o1<o2) {
                laptops[o1pos]=laptops[o1pos]+laptops[o2pos];
                                                no_of_laptops--;
              cout<<"Products Successfully Combined."<<endl;
                            cout<<"Product ID: "<<o1<<endl;
                                          } else {
          laptops[o2pos]=laptops[o1pos]+laptops[o2pos];
                                                no_of_laptops--;
              cout<<"Products Successfully Combined."<<endl;
                              cout<<"Product ID: "<<o2<<endl;
                                          }
                                    } else
                      cout<<"Product Names don't match."<<endl;
                              } else
                      cout<<"Product Names don't match."<<endl;
                              }
                              }
                    } else
                              cout<<"Product ID not found."<<endl;
                } else
                    cout<<"Product ID not found."<<endl;
          }
```

```
                    } else {
                        cout<<"Type of Cloth:\n\t1. Shirt\n\t2. Pant"<<endl;
                        cout<<"Your Choice: ";
                        cin>>category;
                        if (category<1 || category>2)
                            break;
                        if (category==1) {
                            cout<<"ID of Product 1: ";
                            cin>>o1;
                            o1pos=search(shirts, no_of_shirts, o1);
                            if (o1pos>=0) {
                                cout<<"ID of Product 2: ";
                                cin>>o2;
                                o2pos=search(shirts, no_of_shirts, o2);
                                                        398,57      50%
                                o2pos=search(shirts, no_of_shirts, o2);
                                if (o2pos>=0) {
                                    if (o1==o2)
                                        cout<<"The IDs are the
same."<<endl;
                                    else {
                                        string
n1=shirts[o1pos].product_name;
                                        string
n2=shirts[o2pos].product_name;
                                        int l1=n1.length();
                                        int l2=n2.length();
                                        if (l1==l2) {
                                            int c=1;
                                            for (int i=0; i<l1; i++) {
                                                if (toupper(n1[i]) !=
toupper(n2[i])) {
                                                    c=0;
                                                    break;
                                                }
                                            }
```

```
                    if (c) {
                        if (o1<o2) {
            shirts[o1pos]=shirts[o1pos]+shirts[o2pos];
                        no_of_shirts--;
cout<<"Products Successfully Combined."<<endl;
            cout<<"Product ID: "<<o1<<endl;
                        } else {
        shirts[o2pos]=shirts[o1pos]+shirts[o2pos];
                        no_of_shirts--;
    cout<<"Products Successfully Combined."<<endl;
            cout<<"Product ID: "<<o2<<endl;
                        }
                    } else
        cout<<"Product Names don't match."<<endl;
                } else
        cout<<"Product Names don't match."<<endl;
            }
        } else
            cout<<"Product ID not found."<<endl;
    } else
        cout<<"Product ID not found."<<endl;
} else {
    cout<<"ID of Product 1: ";
    cin>>o1;
    o1pos=search(pants, no_of_pants, o1);
    if (o1pos>=0) {
        cout<<"ID of Product 2: ";
        cin>>o2;
        o2pos=search(pants, no_of_pants, o2);
        if (o2pos>=0) {
            if (o1==o2)
            cout<<"The IDs are the same."<<endl;
```

```
                    else {
                        string n1=pants[o1pos].product_name;
                        string n2=pants[o2pos].product_name;
                            int l1=n1.length();
                            int l2=n2.length();
                            if (l1==l2) {
                                int c=1;
                                for (int i=0; i<l1; i++) {
                                if (toupper(n1[i]) != toupper(n2[i])) {
                                        c=0;
                                        break;
                                    }
                                }
                                if (c) {
                                    if (o1<o2) {
                                pants[o1pos]=pants[o1pos]+pants[o2pos];
                                        no_of_pants--;
                    cout<<"Products Successfully Combined."<<endl;
                            cout<<"Product ID: "<<o1<<endl;
                                    } else {
                                pants[o2pos]=pants[o1pos]+pants[o2pos];
                                        no_of_pants--;
                    cout<<"Products Successfully Combined."<<endl;
                                cout<<"Product ID: "<<o2<<endl;
                                    }
                                } else
                                cout<<"Product Names don't match."<<endl;
                            } else
                                cout<<"Product Names don't match."<<endl;
                            }
                        } else
                            cout<<"Product ID not found."<<endl;
                    } else
                        cout<<"Product ID not found."<<endl;
                }
            }
            break;
```

```
case 9:
        cout<<"\nCustomer Phone Number: ";
        cin>>cphone;
    cpos=searchCustomer(customers, no_of_customers, cphone);
        if (cpos>=0) {
       cout<<"Product Category:\n\t1. Gadget\n\t2. Cloth"<<endl;
            cout<<"Your Choice: ";
            cin>>category;
            if (category<1 || category>2)
                break;
            if (category==1) {
         cout<<"Type of Device:\n\t1. Phone\n\t2. Laptop"<<endl;
                cout<<"Your Choice: ";
                cin>>category;
                cout<<"Product ID: ";
                cin>>pr_id;
                if (category<1 || category>2)
                    break;
                 if (category==1) {
                    ppos=search(phones, no_of_phones, pr_id);
                    if (ppos>=0) {
                        cout<<"Quantity: ";
                        cin>>qty;
                        Phone p = phones[ppos];
                        if (qty<=p.quantity) {
                            p.quantity=qty;
                            phones[ppos].quantity-=qty;
                            Bill b;
                            b.createBill(b_id);
                            b_id++;
                            b.getCustomer(customers[cpos]);
                            b.add(p);
                cout<<"Bill Created\nBill ID: "<<b.bill_id<<endl;
                            bills[no_of_bills]=b;
                            no_of_bills++;
                        } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                    } else
                        cout<<"Product not found."<<endl;
                }
```

```
                    } else {
                        ppos=search(laptops, no_of_laptops, pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Laptop p=laptops[ppos];
                            if (qty<=p.quantity) {
                                p.quantity=qty;
                                laptops[ppos].quantity-=qty;
                                Bill b;
                                b.createBill(b_id);
                                b_id++;
                                b.getCustomer(customers[cpos]);
                                b.add(p);
                                cout<<"Bill Created\nBill ID:
"<<b.bill_id<<endl;
                                bills[no_of_bills]=b;
                                no_of_bills++;
                            } else
                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                        } else
                            cout<<"Product not found."<<endl;
                    }
                } else {
                    cout<<"Type of Cloth:\n\t1. Shirt\n\t2.
Pant"<<endl;
                    cout<<"Your Choice: ";
                    cin>>category;
                    if (category<1 || category>2)
                        break;
                    cout<<"Product ID: ";
                    cin>>pr_id;
                    if (category==1) {
                        ppos=search(shirts, no_of_shirts, pr_id);
                        if (ppos>=0) {
                            cout<<"Quantity: ";
                            cin>>qty;
                            Shirt p=shirts[ppos];
```

```
                    if (qty<=p.quantity) {
                        p.quantity=qty;
                        shirts[ppos].quantity-=qty;
                        Bill b;
                        b.createBill(b_id);
                        b_id++;
                        b.getCustomer(customers[cpos]);
                        b.add(p);
                cout<<"Bill Created\nBill ID: "<<b.bill_id<<endl;
                        bills[no_of_bills]=b;
                        no_of_bills++;
                    } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                } else
                    cout<<"Product not found."<<endl;
            } else {
                ppos=search(pants, no_of_pants, pr_id);
                if (ppos>=0) {
                    cout<<"Quantity: ";
                    cin>>qty;
                    Pant p=pants[ppos];
                    if (qty<=p.quantity) {
                        p.quantity=qty;
                        pants[ppos].quantity-=qty;
                        Bill b;
                        b.createBill(b_id);
                        b_id++;
                        b.getCustomer(customers[cpos]);
                        b.add(p);
            cout<<"Bill Created\nBill ID: "<<b.bill_id<<endl;
                        bills[no_of_bills]=b;
                        no_of_bills++;
                    } else
cout<<"Required quantity of "<<p.product_name<<" is not available."<<endl;
                } else
                    cout<<"Product not found."<<endl;
            }}
        } else
            cout<<"Customer not found."<<endl;
        break;
```

```
                case 10:
                        cout<<"\nBill ID: ";
                        cin>>id;
                        pos=searchBill(bills, no_of_bills, id);
                        if (pos>=0) {
                                cout<<"Product Category:\n\t1. Gadget\n\t2.
Cloth"<<endl;
                                cout<<"Your Choice: ";
                                cin>>category;
                                if (category<1 || category>2)
                                        break;
                                if (category==1) {
                                        cout<<"Type of Device:\n\t1. Phone\n\t2.
Laptop"<<endl;
                                        cout<<"Your Choice: ";
                                        cin>>category;
                                        cout<<"Product ID: ";
                                        cin>>pr_id;
                                        if (category<1 || category>2)
                                                break;
                                         if (category==1) {
                                                ppos=search(phones, no_of_phones, pr_id);
                                                if (ppos>=0) {
                                                        cout<<"Quantity: ";
                                                        cin>>qty;
                                                        Phone p = phones[ppos];
                                                        if (qty<=p.quantity) {
                                                                p.quantity=qty;
                                                                phones[ppos].quantity-=qty;
                                                                bills[pos].add(p);
                                                        } else
                                                                cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                                                } else
                                                        cout<<"Product not found."<<endl;
                                        } else {
                                                ppos=search(laptops, no_of_laptops, pr_id);
                                                if (ppos>=0) {
                                                        cout<<"Quantity: ";
                                                        cin>>qty;
```

```
                                        Laptop p=laptops[ppos];
                                        if (qty<=p.quantity) {
                                            p.quantity=qty;
                                            laptops[ppos].quantity-=qty;
                                            bills[pos].add(p);
                                        } else
                                            cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                                    } else
                                        cout<<"Product not found."<<endl;
                                }
                            } else {
                                cout<<"Type of Cloth:\n\t1. Shirt\n\t2.
Pant"<<endl;
                                cout<<"Your Choice: ";
                                cin>>category;
                                if (category<1 || category>2)
                                    break;
                                cout<<"Product ID: ";
                                cin>>pr_id;
                                if (category==1) {
                                    ppos=search(shirts, no_of_shirts, pr_id);
                                    if (ppos>=0) {
                                        cout<<"Quantity: ";
                                        cin>>qty;
                                        Shirt p=shirts[ppos];
                                        if (qty<=p.quantity) {
                                            p.quantity=qty;
                                            shirts[ppos].quantity-=qty;
                                            bills[pos].add(p);
                                        } else
                                            cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                                    } else
                                        cout<<"Product not found."<<endl;
                                } else {
                                    ppos=search(pants, no_of_pants, pr_id);
                                    if (ppos>=0) {
                                        cout<<"Quantity: ";
                                        cin>>qty;
```

```
                                        Pant p=pants[ppos];
                                        if (qty<=p.quantity) {
                                            p.quantity=qty;
                                            pants[ppos].quantity-=qty;
                                            bills[pos].add(p);
                                        } else
                                            cout<<"Required quantity of
"<<p.product_name<<" is not available."<<endl;
                                    } else
                                        cout<<"Product not found."<<endl;
                                }
                            }
                        } else
                            cout<<"Bill ID not found."<<endl;
                        break;
                    case 11:
                         cout<<"\nID of Bill 1: ";
                        cin>>o1;
                        o1pos=searchBill(bills, no_of_bills, o1);
                        if (o1pos>=0) {
                            cout<<"ID of Bill 2: ";
                            cin>>o2;
                            o2pos=searchBill(bills, no_of_bills, o2);
                            if (o2pos>=0) {
                                if (o1==o2)
                                    cout<<"The IDs are the same."<<endl;
                                else {
                                    string c1=bills[o1pos].customer.phone_no;
                                    string c2=bills[o2pos].customer.phone_no;
                                    if (c1==c2) {
                                        np1=bills[o1pos].no_of_phones;
                                        np2=bills[o2pos].no_of_phones;
                                        t1=np1+np2;
```

```
                    for (int i=0; i<np1; i++)
                        for (int j=0; i<np2; j++)
if (bills[o1pos].laptops[i].product_id==bills[o2pos].laptops[j].product_id)
                            t2--;
                    np1=bills[o1pos].no_of_shirts;
                    np2=bills[o2pos].no_of_shirts;
                    t3=np1+np2;
                    for (int i=0; i<np1; i++)
                        for (int j=0; i<np2; j++)
if (bills[o1pos].shirts[i].product_id==bills[o2pos].shirts[j].product_id)
                            t3--;
                    np1=bills[o1pos].no_of_pants;
                    np2=bills[o2pos].no_of_pants;
                    t4=np1+np2;
                    for (int i=0; i<np1; i++)
                        for (int j=0; i<np2; j++)
if (bills[o1pos].pants[i].product_id==bills[o2pos].pants[j].product_id)
                            t4--;
                if (t1<=3 && t2<=3 && t3<=3 && t4<=3) {
                        if (o1<o2) {
                bills[o1pos]=bills[o1pos]+bills[o2pos];
                for (int i=o2pos; i<no_of_bills-1; i++)
                            bills[i]=bills[i+1];
                        no_of_bills--;
            cout<<"Bills Successfully Combined."<<endl;
            cout<<"Bill ID: "<<o1<<endl;
                        } else {
                bills[o2pos]=bills[o2pos]+bills[o1pos];
                for (int i=o1pos; i<no_of_bills-1; i++)
                            bills[i]=bills[i+1];
                        no_of_bills--;
            cout<<"Bills Successfully Combined."<<endl;
            cout<<"Bill ID: "<<o2<<endl;
                        }
                    } else
                        cout<<"These bills can't be
combined as the product count exceeds the assigned limit."<<endl;
                } else
                    cout<<"These bills can't be combined as
they belong to different customers."<<endl;
```

```
                        }
                    } else
                        cout<<"ID not found."<<endl;
                } else
                    cout<<"ID not found."<<endl;
                break;
            case 12:
                cout<<"\nBill ID: ";
                cin>>id;
                pos=searchBill(bills, no_of_bills, id);
                if (pos>=0)
                    bills[pos].displayBill();
                else
                    cout<<"Bill ID not found."<<endl;
                break;
            default:
                cout<<"Invalid Choice."<<endl;
                break;
        }
    }
    return 0;
}
```

RESULT

A supermarket management application system was successfully implemented using object oriented programming concepts.

Exp. No: 6
Date: 12/03/2024

IMPLEMENTATION OF STACK, QUEUE, AND LIST DATA STRUCTURES USING STL CONCEPTS

AIM

To implement stack, queue, and list data structures using Standard Template Library (STL) concepts.

ALGORITHM

1) Stack

Name	Functionality
empty()	Returns whether the stack is empty
size()	Returns size of the stack
top()	Returns the topmost element of the stack
push(e)	Adds element *e* to the top of the stack
pop()	Deletes the topmost element of the stack

2) Queue

Name	Functionality
empty()	Returns whether the queue is empty
size()	Returns size of the queue
front()	Returns the first element of the queue
back()	Returns the last element of the queue
push(e)	Adds element *e* to the end of the queue
pop()	Deletes the first element of the queue

3) List

Name	Functionality
empty()	Returns whether the list is empty
size()	Returns size of the list
front()	Returns the first element of the list
back()	Returns the last element of the list
push_front(e)	Adds element *e* to the beginning of the list
push_back(e)	Adds element *e* to the end of the list
pop_front()	Deletes first element of the list
pop_back()	Deletes last element of the list
remove(e)	Removes all occurences of *e* from the list
sort()	Sorts the list in ascending order
reverse()	Reverses the list
clear()	Removes all elements from the list

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_05$ g++ stack.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_05$ ./a.out
Implementation of Stack

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 1

Element to be Pushed: 7
7 has been pushed onto the stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 1

Element to be Pushed: 6
6 has been pushed onto the stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 1
```

SOURCE CODE

Stack

```
#include <iostream>
#include <stack>
using namespace std;

template <typename T>
void printStack(stack<T> s) {
	stack<T> temp;
	int e;
	while (!s.empty()) {
		e=s.top();
		s.pop();
		temp.push(e);
		cout<<e<<" ";
	}
	cout<<endl;
	while (!temp.empty()) {
		e=temp.top();
		temp.pop();
		s.push(e);
	}
}

int main() {
	stack<int> s;
	int choice, e;
	cout<<"Implementation of Stack"<<endl;
	while (1) {
		cout<<"\n0. Exit\n1. Push\n2. Pop\n3. Peek\n4. Size\n5.
Print"<<endl;
		cout<<"Your Choice: ";
		cin>>choice;
		if (choice==0)
			break;
		switch (choice) {
			case 1:
				cout<<"\nElement to be Pushed: ";
```

```
Element to be Pushed: 5
5 has been pushed onto the stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 3

Top of the Stack: 5

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 4

Size of the Stack: 3

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 2

5 has been popped from the stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 5
```

```
Stack Elements: 6 7

0. Exit
1. Push
2. Pop
3. Peek
4. Size
5. Print
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_05$
```

```
                cin>>e;
                s.push(e);
                cout<<e<<" has been pushed onto the stack."<<endl;
                break;
            case 2:
                if (!s.empty()) {
                    e=s.top();
                    s.pop();
                    cout<<"\n"<<e<<" has been popped from the
stack."<<endl;
                } else
                    cout<<"\nStack is empty."<<endl;
                break;
            case 3:
                if (!s.empty())
                    cout<<"\nTop of the Stack: "<<s.top()<<endl;
                else
                    cout<<"\nStack is empty."<<endl;
                break;
            case 4:
                cout<<"\nSize of the Stack: "<<s.size()<<endl;

                break;
            case 5:
                if (!s.empty()) {
                    cout<<"\nStack Elements: ";
                    printStack<int>(s);
                } else
                    cout<<"\nStack is empty."<<endl;
                break;
            default:
                cout<<"Invalid Choice."<<endl;
                break;
        }
    }
    return 0;
}
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_05$ g++ queue.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_05$ ./a.out
Implementation of Queue

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 1

Element to be Enqueued: 7
7 has been successfully enqueued.

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 1

Element to be Enqueued: 6
6 has been successfully enqueued.

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 1

Element to be Enqueued: 5
5 has been successfully enqueued.
```

Queue

```
#include <iostream>
#include <queue>
using namespace std;

void printQueue (queue<int> q) {
	queue<int> temp;
	int e;
	while (!q.empty()) {
		e=q.front();
		q.pop();
		temp.push(e);
		cout<<e<<" ";
	}
	cout<<endl;
	while (!temp.empty()) {
		e=temp.front();
		temp.pop();
		q.push(e);
	}
}
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 3

Front of the Queue: 7

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 4

Back of the Queue: 5

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 5

Size of the Queue: 3

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 6
```

```
int main() {
	queue<int> q;
	int choice, e;
	cout<<"Implementation of Queue"<<endl;
	while (1) {
		cout<<"\n0. Exit\n1. Enqueue\n2. Dequeue\n3. Check Front\n4.
Check Rear\n5. Size\n6. Print"<<endl;
		cout<<"Your Choice: ";
		cin>>choice;
		if (choice==0)
			break;
		switch (choice) {
			case 1:
				cout<<"\nElement to be Enqueued: ";
				cin>>e;
				q.push(e);
				cout<<e<<" has been successfully enqueued."<<endl;
				break;
			case 2:
				if (!q.empty()) {
					e=q.front();
					q.pop();
					cout<<"\n"<<e<<" has been successfully
dequeued."<<endl;
				} else
					cout<<"\nQueue is Empty."<<endl;
				break;
			case 3:
				if (!q.empty()) {
					cout<<"\nFront of the Queue: "<<q.front()<<endl;
				} else
					cout<<"\nQueue is Empty."<<endl;
				break;
			case 4:
				if (!q.empty()) {
					cout<<"\nBack of the Queue: "<<q.back()<<endl;
				} else
					cout<<"\nQueue is Empty."<<endl;
				break;
```

```
Queue Elements: 7 6 5

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 2

7 has been successfully dequeued.

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 6

Queue Elements: 6 5

0. Exit
1. Enqueue
2. Dequeue
3. Check Front
4. Check Rear
5. Size
6. Print
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_05$
```

```
                case 5:
                        cout<<"\nSize of the Queue: "<<q.size()<<endl;
                        break;
                case 6:
                        if (!q.empty()) {
                                cout<<"\nQueue Elements: ";
                                printQueue(q);
                        } else
                                cout<<"\nQueue is Empty."<<endl;
                        break;
                default:
                        cout<<"\nInvalid Choice."<<endl;
                        break;
            }
        }
        return 0;
}
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_05$ g++ list.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_05$ ./a.out
Implementation of List

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 1

Element to be Pushed: 1
1 has been successfully pushed into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 1

Element to be Pushed: 2
2 has been successfully pushed into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 2

Element to be Appended: 3
3 has been successfully appended into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 2

Element to be Appended: 3
3 has been successfully appended into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 2

Element to be Appended: 4
4 has been successfully appended into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 2

Element to be Appended: 4
4 has been successfully appended into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 2

Element to be Appended: 5
5 has been successfully appended into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 1

Element to be Pushed: 6
6 has been successfully pushed into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 1

Element to be Pushed: 7
7 has been successfully pushed into the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 1

Element to be Pushed: 7
7 has been successfully pushed into the list.
```

List

```
#include <iostream>
#include <iterator>
#include <list>
using namespace std;

void printList(list<int> l) {
	list<int>::iterator i;
	for (i=l.begin(); i!=l.end(); i++)
		cout<<*i<<" ";
	cout<<endl;
}

int main() {
	list<int> l;
	int choice, e, pos;
	cout<<"Implementation of List"<<endl;
	while (1) {
		cout<<"\n0. Exit
			\n1. Push
			\n2. Append
			\n3. Peek Front
			\n4. Peek Back
			\n5. Length
			\n6. Delete
			\n7. Pop
			\n8. Sort
			\n9. Reverse
			\n10. Remove
			\n11. Print
			\n12. Clear"<<endl;

		cout<<"Your Choice: ";
		cin>>choice;
		if (choice==0)
			break;
```

```
0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 12

List Elements: 7 7 6 2 1 3 3 4 4 5

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 3

Front of the List: 7

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 4

Back of the List: 5

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 5

Length of the List: 10
```

```
0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 6

7, the first element, has been deleted from the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 7

5, the last element, has been popped from the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 12

List Elements: 7 6 2 1 3 3 4 4

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 8

Value to be Searched: 6
6 is present in position 1 of the list.
```

```
switch(choice) {
        case 1:
                cout<<"\nElement to be Pushed: ";
                cin>>e;
                l.push_front(e);
            cout<<e<<" has been successfully pushed into the list."<<endl;
                break;
        case 2:
                cout<<"\nElement to be Appended: ";
                cin>>e;
                l.push_back(e);
       cout<<e<<" has been successfully appended into the list."<<endl;
                break;
        case 3:
                if (!l.empty())
                        cout<<"\nFront of the List: "<<l.front()<<endl;
                else
                        cout<<"\nList is empty."<<endl;
                break;
        case 4:
                if (!l.empty())
                        cout<<"\nBack of the List: "<<l.back()<<endl;
                else
                        cout<<"\nList is empty."<<endl;
                break;
        case 5:
                cout<<"\nLength of the List: "<<l.size()<<endl;
                break;
        case 6:
                if (!l.empty()) {
                        e=l.front();
                        l.pop_front();
cout<<"\n"<<e<<", the first element, has been deleted from the list."<<endl;
                } else
                        cout<<"\nList is empty."<<endl;
                break;
```

```
0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 9
Current List: 7 6 2 1 3 3 4 4
Sorted List: 1 2 3 3 4 4 6 7

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 10
Current List: 1 2 3 3 4 4 6 7
Reversed List: 7 6 4 4 3 3 2 1
```

```
0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 11

Element to be Removed: 4
All occurrences of 4 have been successfully removed from the list.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 12

List Elements: 7 6 3 3 2 1
```

```
                case 7:
                        if (!l.empty()) {
                                e=l.back();
                                l.pop_back();
        cout<<"\n"<<e<<", the last element, has been popped from the list."<<endl;
                        } else
                                cout<<"\nList is empty."<<endl;
                        break;
                case 8:
                        if (!l.empty()){
                                cout<<"Current List: ";
                                printList(l);
                                l.sort();
                                cout<<"Sorted List: ";
                                printList(l);
                        } else
                                cout<<"\nList is empty."<<endl;
                        break;
                case 9:
                        if (!l.empty()) {
                                cout<<"Current List: ";
                                printList(l);
                                l.reverse();
                                cout<<"Reversed List: ";
                                printList(l);
                        } else
                                cout<<"\nList is empty."<<endl;
                        break;
                case 10:
                        if (!l.empty()) {
                                cout<<"\nElement to be Removed: ";
                                cin>>e;
                                l.remove(e);
                                cout<<"All occurrences of "<<e<<" have been
successfully removed from the list."<<endl;
                        } else
                                cout<<"\nList is empty."<<endl;
                        break;
```

```
0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 13

List has been cleared.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 12

List is empty.
```

```
List is empty.

0. Exit
1. Push
2. Append
3. Peek Front
4. Peek Back
5. Length
6. Delete
7. Pop
8. Search
9. Sort
10. Reverse
11. Remove
12. Print
13. Clear
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_05$
```

```
                case 11:
                        if (!l.empty()) {
                                cout<<"\nList Elements: ";
                                printList(l);
                        } else
                                cout<<"\nList is empty."<<endl;
                        break;
                case 12:
                        if (!l.empty()) {
                                l.clear();
                                cout<<"\nList has been cleared."<<endl;
                        } else
                                cout<<"\nList is empty."<<endl;
                        break;
                default:
                        cout<<"\nInvalid Choice."<<endl;
                        break;
            }
        }
        return 0;
}
```

RESULT

Stack, queue, and list data structures were successfully implemented using Standard Template Library concepts.

Exp. No: 7
Date: 19/03/2024

IMPLEMENTATION OF AVL TREE USING TEMPLATES

AIM

To implement Adelson-Velsky and Landis (AVL) Tree insertion, balancing, deletion, search, and traversal using templates.

ALGORITHM

```
AVL {

    Node
        data, left, right, height
        data is of type T
        Node (newdata)
            height=1
            data=newdata
            left=NULL
            right=NULL

    rightRotate (root)
        new = left of root
        left of root = right of new
        right of new = root
        calculate height of root
        calculate height of new
        return new

    leftRotate (root)
        new = right of root
        right of root = left of new
        left of new = root
        calculate height of root
        calculate height of new
        return new
```

```
height (root)
        if root is NULL return 0
        return root->height

inorder (root)
        if root is NULL return
        inorder (left of root)
        display data in root
        inorder (right of root)

preorder (root)
        if root is NULL return
        display data in root
        preorder (left of root)
        preorder (right of root)

postorder (root)
        if root is NULL return
        postorder (left of root)
        postorder (right of root)
        display data in root

insert (root, data)
        if root is NULL
                new Node (data)
                return new Node
        if (data is less than data in root) insert (left of root, data)
        else if (data is greater than data in root) insert (right of root, data)

        calculate height of root
        balance factor bf = height (left of root) – height (right of root)

        if bf > 1
                if data is less than left of data in root
                        return rightRotate (root)
                else
                        left of root = leftRotate (left of root)
                        return rightRotate(root)
```

```
        else if bf < -1
                if data is greater than right of data in root
                        return leftRotate (root)
                else
                        right of root =rightRotate (root)
                        return leftRotate (root)
        return root

inorderSuccessor (root)
        if (root is NULL) return root
        if left of root is NOT NULL return inorderSuccessor (left of root)
        return root

delete (root, data)
        if root is NULL return root
        if data is less than data in root
                left of root = delete (left of root, data)
        else if data is greater than data in root
                right of root = delete (right of root, data)
        else
                if left of root and right of root are both NULL
                        root=NULL
                else if left of root is NULL
                        data in root = right of data in root
                        right of root = delete (right of root, data)
                else if right of root is NULL
                        data in root = left of data in root
                        left of root = delete (left of root, data)
                else
                        is = inorderSuccessor (right of root)
                        data in root = data in is
                        right of root = delete (right of root, is->data)

        if root is NULL return root;
        calculate height of root

        balance factor bf = height (left of root) – height (right of root)
```

```
        if bf >1
                if height of left of left of root is greater than or equal to
                    height of left of right of root, then
                        return rightRotate (root)
                else
                        left of root = leftRotate (root)
                        return rightRotate (root)
        else if bf < -1
                if height of right of right of root is greater than or equal to
                   height of right of left of root, then
                        return leftRotate (root)
                else
                        right of root = rightRotate (root)
                        return leftRotate (root)
        return root

search (root, data)
        if root is not NULL
                if data is equal to data in root
                        return 1
                else if data is less than data in root
                        return search (left of root, data)
                else if data is greater than data in root
                        return search (right of root, data)
        return 0
}
```

SOURCE CODE

```
#include<iostream>
using namespace std;

// Insert 21, 26, 33, 7, 3, 12, 30
// Delete 3, 7, 12

template <typename T>
class AVL {
	public:
		class Node {
			public:
				T data;
				int height;
				Node * left;
				Node * right;
				Node(T k) {
					height = 1;
					data = k;
					left = NULL;
					right = NULL;
				}
		};
		int n=0;
		Node * root = NULL;
		void insert (T x) {
			root=insertNode(root, x);
		}
		void remove (T x) {
			root=deleteNode(root, x);
		}
		int search(T x) {
			return searchNode(root, x);
		}
		void inorder() {
			inorderTraversal(root);
			cout<<endl;
		}
```

```
		void preorder() {
			preorderTraversal(root);
			cout<<endl;
		}
		void postorder() {
			postorderTraversal(root);
			cout<<endl;
		}
	private:
		int height(Node * root) {
			if(root==NULL) return 0;
			return root->height;
			}
		Node * rightRotation(Node * root) {
			Node * newroot = root->left;
			root->left = newroot->right;
			newroot->right = root;
			root->height = 1+max(height(root->left),
height(root->right));
			newroot->height = 1+max(height(newroot->left),
height(newroot->right));
			return newroot;
		}
		Node * leftRotation(Node * root) {
			Node * newroot = root->right;
			root->right = newroot->left;
			newroot->left = root;
			root->height = 1+max(height(root->left),
height(root->right));
			newroot->height = 1+max(height(newroot->left),
height(newroot->right));
			return newroot;
		}
		void inorderTraversal(Node * root) {
			if (root==NULL) return;
			inorderTraversal(root->left);
			cout<<root->data<<" ";
			inorderTraversal(root->right);
		}
```

```
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_06$ g++ avl.cpp
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_06$ ./a.out

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1

Element: 21
Inserting 21 in the AVL Tree...
21 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1

Element: 26
Inserting 26 in the AVL Tree...
26 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1
```

```
void preorderTraversal(Node * root) {
	if (root==NULL) return;
	cout<<root->data<<" ";
	preorderTraversal(root->left);
	preorderTraversal(root->right);
}
void postorderTraversal(Node * root) {
	if (root==NULL) return;
	postorderTraversal(root->left);
	postorderTraversal(root->right);
	cout<<root->data<<" ";
}
Node * insertNode(Node * root, T x) {
	if(root==NULL) {
		n+=1;
		Node * temp = new Node(x);
		return temp;
	}
	if (x < root->data) root->left = insertNode(root->left, x);
	else if(x > root->data) root->right = insertNode(root->right,
x);
	root->height = 1 + max(height(root->left),
height(root->right));
	int bal = height(root->left)-height(root->right);
	if (bal>1) {
		if(x < root->left->data) {
			cout<<"Right Rotating the AVL Tree..."<<endl;
			return rightRotation(root);
		} else {
			cout<<"Left Right Rotating the AVL
Tree..."<<endl;
			root->left = leftRotation(root->left);
			return rightRotation(root);
		}
	} else if(bal<-1) {
		if (x > root->right->data) {
			cout<<"Left Rotating the AVL Tree..."<<endl;
			return leftRotation(root);
```

```
Element: 33
Inserting 33 in the AVL Tree...
Left Rotating the AVL Tree...
33 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1

Element: 7
Inserting 7 in the AVL Tree...
7 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1

Element: 3
Inserting 3 in the AVL Tree...
Right Rotating the AVL Tree...
3 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1
```

```
                        } else {
                                cout<<"Right Left Rotating the AVL
Tree..."<<endl;
                                root->right = rightRotation(root->right);
                                return leftRotation(root);
                        }
                }
                return root;
        }
        Node * findInorderSuccessor(Node * root) {
                if (root==NULL) return root;
                if (root->left!=NULL)
                        return findInorderSuccessor(root->left);
                return root;
        }
        Node * deleteNode(Node * root, T x) {
                if (root==NULL) return NULL;
                if (x<root->data) {
                        root->left=deleteNode(root->left, x);
                } else if (x>root->data) {
                        root->right=deleteNode(root->right, x);
                } else {
                        if (root->left==NULL && root->right==NULL) {
                                root=NULL;
                        } else if (root->left==NULL) {
                                root->data=root->right->data;
                                root->right=deleteNode(root->right,
root->right->data);
                        } else if (root->right==NULL) {
                                root->data=root->left->data;
                                root->left=deleteNode(root->left,
root->left->data);
                        } else {
                                Node * inorderSuccessor =
findInorderSuccessor(root->right);
                                root->data=inorderSuccessor->data;
                                root->right=deleteNode(root->right,
inorderSuccessor->data);
                        }
                }
```

```
Element: 12
Inserting 12 in the AVL Tree...
Left Right Rotating the AVL Tree...
12 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 1

Element: 30
Inserting 30 in the AVL Tree...
Right Left Rotating the AVL Tree...
30 has been successfully inserted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 4

Inorder Traversal of AVL Tree...
3 7 12 21 26 30 33

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 5
```

```
		if (root==NULL) return root;
		root->height = 1 + max(height(root->left),
height(root->right));
		int bal = height(root->left)-height(root->right);
		if (bal>1) {
			if (height(root->left->left) >=
height(root->left->right)) {
				cout<<"Right Rotating the AVL Tree..."<<endl;
				return rightRotation(root);
			} else {
				cout<<"Left Right Rotating the AVL
Tree..."<<endl;
				root->left = leftRotation(root->left);
				return rightRotation(root);
			}
		} else if(bal<-1) {
			 if (height(root->right->right) >=
height(root->right->left)) {
				cout<<"Left Rotating the AVL Tree..."<<endl;
				return leftRotation(root);
			} else {
			cout<<"Right Left Rotating the AVL Tree..."<<endl;
				root->right = rightRotation(root->right);
				return leftRotation(root);
			}
		}
		return root;
	}
	int searchNode (Node * root, T x) {
		if(root!=NULL) {
			if(x==root->data)
				return 1;
			else if (x<root->data)
				return searchNode(root->left, x);
			else if (x>root->data)
				return searchNode(root->right, x);
		}
		return 0;
	}
};
```

```
Preorder Traversal of AVL Tree...
21 7 3 12 30 26 33

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 6

Postorder Traversal of AVL Tree...
3 12 7 26 33 30 21

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 3

Element: 3
3 has been successfully deleted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 3

Element: 7
7 has been successfully deleted.
```

```
int main() {
	int choice, e;
	AVL<int> tree;
	while (1) {
		cout<<"\n0. Exit\n1. Insert\n2. Search\n3. Delete\n4. Inorder\n5.
Preorder\n6. Postorder"<<endl;
		cout<<"Your Choice: ";
		cin>>choice;
		if (choice==0) break;
		switch (choice) {
			case 1:
				cout<<"\nElement: ";
				cin>>e;
				if (tree.search(e))
			     cout<<e<<" is already present in the AVL Tree."<<endl;
				else {
				cout<<"Inserting "<<e<<" in the AVL Tree..."<<endl;
					tree.insert(e);
				cout<<e<<" has been successfully inserted."<<endl;
				}
				break;
			case 2:
				cout<<"\nElement: ";
				cin>>e;
				if (tree.search(e))
				cout<<e<<" is present in the AVL Tree."<<endl;
				else
				cout<<e<<" is NOT present in the AVL Tree."<<endl;
				break;
			case 3:
				cout<<"\nElement: ";
				cin>>e;
				if (tree.search(e)) {
					tree.remove(e);
				cout<<e<<" has been successfully deleted."<<endl;
				} else
				cout<<e<<" is NOT present in the AVL Tree."<<endl;
				break;
```

```
0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 3

Element: 12
Left Rotating the AVL Tree...
12 has been successfully deleted.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 5

Preorder Traversal of AVL Tree...
30 21 26 33

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++_programs/exp_06$
```

```
                case 4:
                        cout<<"\nInorder Traversal of AVL Tree..."<<endl;
                        tree.inorder();
                        break;
                case 5:
                        cout<<"\nPreorder Traversal of AVL Tree..."<<endl;
                        tree.preorder();
                        break;
                case 6:
                        cout<<"\nPostorder Traversal of AVL Tree..."<<endl;
                        tree.postorder();
                        break;
                default:
                        cout<<"\nInvalid Choice."<<endl;
                        break;
            }
        }
}
```

RESULT

AVL Tree insertion, search, deletion, balancing, and traversal were all successfully implemented using templates.

Exp. No: 8
Date: 26/03/2024

IMPLEMENTATION OF SPLAY TREE USING TEMPLATES

AIM

To implement Splay Tree insertion, deletion, and search using templates.

ALGORITHM

```
Splay Tree {

    Splay
        data, left, right
        data is of type T

    root=NULL

    Zig Rotation (root)
        new = left of root
        left of root = right of new
        right of new = root
        calculate height of root
        calculate height of new
        return new

    Zag Rotation (root)
        new = right of root
        right of root = left of new
        left of new = root
        calculate height of root
        calculate height of new
        return new
```

```
splay (data)
        if root is NULL return root
        Node header
        left of header = right of header = NULL
        ltm = &header
        rtm = &header
        while Splay is not complete
                if data is less than data in root
                        if left of root is NULL exit
                        if data is less than data in left of root
                                Zig Rotation (root)
                                if left of root is NULL exit
                        left of rtm = root
                        rtm = left of rtm
                        root = left of root
                        left of rtm = NULL
                else if (data is greater than data in root)
                        if right of root is NULL exit
                        if data is greater than data in right of root
                                Zag Rotation (root)
                                if right of root is NULL exit
                        right of ltm = root
                        ltm = right of ltm
                        root = right of root
                        right of ltm = NULL
                else exit
        right of ltm = left of root
        left of rtm = right of root
        left of root = right of header
        right of root = left of header
        return root

insert (data)
        static splay p
        if p is NULL
                p = new Splay (data)
        else
                data in p = data
```

```
            if root is NULL
                root = p
                p=NULL
                return root

            splay (data)

            if data is less than data in root
                left of p = left of root
                right of p = root
                left of root = NULL
                root = p
            else if data is greater than data in root
                right of p = right of root
                left of p = root
                right of root = NULL
                root = p
            else
                return root
            p = NULL
            return root

delete (data)
    if data in Splay Tree
            if root is NULL
                return root
            splay (data)
            if data is equal to data in root
                if left of root is NULL
                    temp = root
                    root = right of root
                else
                    temp = root
                    splay (data) → with respect to left of root
                delete temp
                return root
            else
                return root
```

```
        search (data)
            if data is in Splay Tree
                return splay (data)
            return root

        preorder ()
            if root is NULL return
            display data in root
            if root has left child
                display data in left of root
            if root has right child
                display data in right of root
            preorder () → wrt to left of root
            preorder () → wrt to right of root
}
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_07$ g++ splay.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_07$ ./a.out

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 1

Value to be Inserted: 10

After Insert: 10
Data: 10

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 1

Value to be Inserted: 3

After Insert: 3
Data: 3 | Right Child: 10
Data: 10

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 1

Value to be Inserted: 20
```

SOURCE CODE

```
#include <iostream>
using namespace std;
template <typename T>
class SplayTree {
        private:
                struct splay {
                        T data;
                        splay* left;
                        splay* right;
                };
                splay* RR_Rotate(splay* k2) {
                        splay* k1=k2->left;
                        k2->left=k1->right;
                        k1->right=k2;
                        return k1;
                }
                splay* LL_Rotate(splay* k2) {
                        splay* k1=k2->right;
                        k2->right=k1->left;
                        k1->left=k2;
                        return k1;
                }
                splay* Splay(T data, splay* root) {
                        if (!root)
                                return NULL;
                splay header;
                        header.left=header.right=NULL;
                        splay* LeftTreeMax=&header;
                        splay* RightTreeMin=&header;
                while (1) {
                        if (data < root->data) {
                                if (!root->left)
                                break;
                                        if (data < root->left->data) {
                                                root=RR_Rotate(root);
                        if (!root->left)
                                                break;
                                        }
```

```
After Insert: 20
Data: 20 | Left Child: 10
Data: 10 | Left Child: 3
Data: 3

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 1

Value to be Inserted: 5

After Insert: 5
Data: 5 | Left Child: 3 | Right Child: 10
Data: 3
Data: 10 | Right Child: 20
Data: 20

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 1

Value to be Inserted: 7

After Insert: 7
Data: 7 | Left Child: 5 | Right Child: 10
Data: 5 | Left Child: 3
Data: 3
Data: 10 | Right Child: 20
Data: 20
```

```
                RightTreeMin->left=root;
                RightTreeMin=RightTreeMin->left;
                    root=root->left;
              RightTreeMin->left=NULL;
            }
        else if (data > root->data) {
                if (!root->right)
                        break;
                    if (data > root->right->data) {
                        root=LL_Rotate(root);
                if (!root->right)
                            break;
                        }
                    LeftTreeMax->right=root;
                    LeftTreeMax=LeftTreeMax->right;
                    root=root->right;
                    LeftTreeMax->right=NULL;
        }
            else
                break;
    }
    LeftTreeMax->right=root->left;
    RightTreeMin->left=root->right;
    root->left=header.right;
    root->right=header.left;
    return root;
}
splay* New_Node(T data) {
        splay* p_node=new splay;
    if (!p_node) {
            return NULL;
        }
    p_node->data=data;
    p_node->left=p_node->right=NULL;
    return p_node;
}
 splay* Insert(T data, splay* root) {
        static splay* p_node=NULL;
    if (!p_node)
            p_node=New_Node(data);
```

```
Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 2

Value to be Deleted: 5

After Delete: 5
Data: 3 | Right Child: 7
Data: 7 | Right Child: 10
Data: 10 | Right Child: 20
Data: 20

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 3

Value to be Searched: 7

After Search 7
Data: 7 | Left Child: 3 | Right Child: 10
Data: 3
Data: 10 | Right Child: 20
Data: 20
```

```
        else
          p_node->data=data;
        if (!root) {
              root=p_node;
          p_node=NULL;
              return root;
        }
        root=Splay(data, root);
        if (data < root->data) {
              p_node->left=root->left;
          p_node->right=root;
              root->left=NULL;
              root=p_node;
        } else if (data > root->data) {
              p_node->right=root->right;
              p_node->left=root;
              root->right=NULL;
              root=p_node;
        } else
              return root;
          p_node=NULL;
        return root;
  }
  int searchNode (T data, splay * root) {
          if (root) {
                  if (root->data==data) return 1;
  if (data<root->data) return searchNode(data, root->left);
  else if (data>root->data) return searchNode(data, root->right);
          }
          return 0;
  }
splay* Delete(T data, splay* root) {
                  splay* temp;
                  if (!root)
                  return NULL;
                  root=Splay(data, root);
                  if (data==root->data) {
                          if (!root->left) {
                                  temp=root;
                                root=root->right;
```

```
Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 2

Value to be Deleted: 10

After Delete: 10
Data: 7 | Left Child: 3 | Right Child: 20
Data: 3
Data: 20

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 3

Value to be Searched: 10

After Search 10
Data: 7 | Left Child: 3 | Right Child: 20
Data: 3
Data: 20

Splay Tree Operations
0. Exit
1. Insert
2. Delete
3. Search

Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_07$
```

```
                        } else {
                                temp=root;
                                  root=Splay(data, root->left);
                                  root->right=temp->right;
                          }
                        delete temp;
                        return root;
                    } else {
                          return root;
                    }
          }
        splay* Search(T data, splay* root) {
                if (searchNode(data, root))
                      return Splay(data, root);
                return root;
        }
        void preOrder(splay * root) {
                if (root) {
                    cout<< "Data: " <<root->data;
                    if(root->left)
                       cout<< " | Left Child: "<<root->left->data;
                    if(root->right)
                       cout << " | Right Child: "<<root->right->data;
                    cout<<endl;
                       preOrder(root->left);
                       preOrder(root->right);
                }
          }
public:
          splay *root=NULL;
          void insert(T data) {
                root=Insert(data, root);
          }
          void remove (T data) {
                if (searchNode(data, root)) {
                       root=Delete(data, root);
                }
          }
          void search (T data) {
                if (searchNode(data, root)) {
```

```
                        root=Search(data, root);
                }
            }
            void preorder() {
                preOrder(root);
            }
};

int main() {
        SplayTree <int> st;
        int e, choice;
        while(1) {
            cout<<"\nSplay Tree Operations\n";
            cout<<"0. Exit\n1. Insert\n2. Delete\n3. Search\n"<<endl;
        cout<<"Your Choice: ";
            cin>>choice;

            if (choice==0)
                break;

            switch (choice) {

                case 1:
                cout<<"\nValue to be Inserted: ";
                    cin>>e;
                    st.insert(e);
                cout<<"\nAfter Insert: "<<e<<endl;
                    st.preorder();
                break;

               case 2:
                cout<<"\nValue to be Deleted: ";
                    cin>>e;
                st.remove(e);
                    cout<<"\nAfter Delete: "<<e<<endl;
                    st.preorder();
                    break;
```

```
                case 3:
                    cout<<"\nValue to be Searched: ";
                    cin>>e;
                    st.search(e);
                    cout<<"\nAfter Search "<<e<<endl;
                    st.preorder();
                    break;
                default:
                    cout<<"\nInvalid Choice.\n";
                      break;
            }
        }
        return 0;
}
```

RESULT

Splay Tree insertion, search, and deletion were all successfully implemented using templates.

Exp. No: 9
Date: 02/04/2024

IMPLEMENTATION OF A HEAP TREE USING TEMPLATES

AIM

To implement Heap Tree insertion, deletion, extraction of top element, and printing using templates.

ALGORITHM

```
Heap {
        initialise heap as a vector
        data type is T
        Heap ()
                push back T(), acts as a placeholder at index 0
        isEmpty()
                if size of heap is 1 return 0
                return 1

        heapifyUp (index)
                parent = index/2
                while index is greater than 1 and heap[index] is less than
heap[parent]
                        swap heap[index] and heap[parent]
                        index = parent
                        parent = index/2

        heapifyDown (index)
                        left = 2*index
                        right = 2*index+1
                        smallest = index
                        if heap[left] is less than heap[smallest]
                                smallest = left
                        if heap[right] is less than heap[smallest]
                                smallest = right
                        if smallest is not equal to index
                                swap heap[index] and heap[smallest]
                                heapifyDown (smallest)
```

```
        insert (value)
                push value into back of the heap
                heapifyUp (size of heap - 1)

        extractTop ()
                top = heap[1]
                swap heap[1] and heap[size of heap – 1]
                pop the last element of the heap
                heapifyDown (1)
                return top

        remove (value)
                find value in heap
                index = position of value in heap
                swap heap[index] and heap[size of heap – 1]
                pop the last element of the heap
                if heap[index] is less than heap[index/2]
                        heapifyUp (index)
                else
                        heapifyDown (index)

        print Heap ()
                for each element e in heap
                        display e
}
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_08$ g++ heap.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_08$ ./a.out

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 11

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 15

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 21

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 30
```

SOURCE CODE

```
#include <iostream>
#include <vector>
#include <algorithm>
// #include <stdexcept>
using namespace std;

template <typename T>
class Heap {
	private:
		vector<T> heap;
	public:
		Heap() {
			heap.push_back(T()); // Placeholder at Index 0
		}
		bool isEmpty() {
			return heap.size()==1;
		}
		void heapifyUp(int index) {
			int parent=index/2;
			while (index>1 && heap[index]<heap[parent]) {
				swap(heap[index], heap[parent]);
				index=parent;
				parent=index/2;
			}
		}
		void heapifyDown(int index) {
			int l=2*index;
			int r=2*index+1;
			int smallest=index;
			if (l<heap.size() && heap[l]<heap[smallest])
				smallest=l;
			if (r<heap.size() && heap[r]<heap[smallest])
				smallest=r;
			if (smallest != index) {
				swap(heap[index], heap[smallest]);
				heapifyDown(smallest);
			}
		}
```

```
0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 7

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 3

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 26

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 3
```

```
        void insert(const T& value) {
            heap.push_back(value);
            heapifyUp(heap.size()-1);
        }

        T extractTop() {
            T top=heap[1];
            swap(heap[1], heap[heap.size()-1]);
            heap.pop_back();
            heapifyDown(1);
            return top;
        }

        void remove(const T& value) {
            auto it=find(heap.begin(), heap.end(), value);
            if (it==heap.end())
                cout<<"Element not found."<<endl;
            int index=distance(heap.begin(), it);
            swap(heap[index], heap[heap.size()-1]);
            heap.pop_back();
            if (index>1 && heap[index]<heap[index/2])
                heapifyUp(index);
            else
                heapifyDown(index);
        }
        void print() {
            for (int i=1; i < heap.size(); i++)
                cout << heap[i] << " ";
            cout << "\n";
        }
};
```

```
0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 1

Value to be Inserted: 33

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 4

Displaying Heap...
3 3 7 11 15 21 26 30 33

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 3

Extracting Top Element...
Top Element = 3

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 2

Value to be Deleted: 15
```

```
int main() {

	Heap<int> ht;
	int e, choice;

	while (1) {

		cout<<"\n0. Exit\n1. Insert\n2. Delete\n3. Extract Top\n4. Print
Heap"<<endl;
		cout<<"Your Choice: ";
		cin>>choice;

		if (choice==0)
			break;
		switch (choice) {

			case 1:
				cout<<"\nValue to be Inserted: ";
				cin>>e;
				ht.insert(e);
				break;

			case 2:
				if (!ht.isEmpty()) {
					cout<<"\nValue to be Deleted: ";
					cin>>e;
					ht.remove(e);
				} else
					cout<<"\nHeap is Empty."<<endl;
				break;

			case 3:
				if (!ht.isEmpty()) {
					cout<<"\nExtracting Top Element..."<<endl;
					cout<<"Top Element = "<<ht.extractTop()<<endl;
				} else
					cout<<"\nHeap is Empty."<<endl;
				break;
```

```
0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 4

Displaying Heap...
3 11 7 30 33 21 26

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 2

Value to be Deleted: 30

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 4

Displaying Heap...
3 11 7 26 33 21

0. Exit
1. Insert
2. Delete
3. Extract Top
4. Print Heap
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_08$
```

```
                case 4:
                        if (!ht.isEmpty()) {
                                cout<<"\nDisplaying Heap..."<<endl;
                                ht.print();
                        } else
                                cout<<"\nHeap is Empty."<<endl;
                        break;
                default:
                        cout<<"\nInvalid Choice."<<endl;
                        break;
            }
        }
    return 0;
}
```

RESULT

Heap Tree insertion, deletion, extraction of top element, and printing were all successfully implemented using templates.

Exp. No: 10
Date: 09/04/2024

IMPLEMENTATION OF GRAPH TRAVERSAL ALGORITHMS: BREADTH-FIRST SEARCH, DEPTH-FIRST SEARCH

AIM

To implement the following graph traversal algorithms:

- breadth-first search
- depth-first search

ALGORITHM

1) Common Functions

```
print (queue)
        while queue is not empty
                e = front of queue
                display e
                dequeue from queue
```

2) Breadth-First Search

```
bfs (adjacency list, source, visited list v)
        initialise a queue q1 for bfs
        initialise a queue q2 for displaying BFS traversal
        v[source] = true
        enqueue source into q1
        while q1 is not empty
                current = front of q1
                dequeue front from q1
                enqueue current into q2
                display current vertex
                for each unvisited neighbour of current vertex
                        enqueue unvisited vertex
                        visited[unvisited vertex] = true
        print (q2)
```

3) Depth-First Search

```
dfs (adjacency list, source, visited list v)
        initialise a stack s for dfs
        initialise a queue q for display
        push source into s
        while stack is not empty
                current = top of s
                pop from s
                enqueue current into q
                visited [current] = true
                for each unvisited neighbour of current vertex
                        push unvisited neighbour into s
        print (q)
```

4) Receiving Graph as Adjacency List

```
get no. of vertices n
initialise adjacency list adj of size n
initialise visited list of size n
receive edges for the user in the format v1, v2
v1, v2 are the vertices
for each valid edge received
        push v2 into adj[v1]
get source vertex
call the bfs/dfs functions, passing adj, source vertex, and visited list
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_09$ g++ bfs.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_09$ ./a.out
No. of Vertices: 7

Vertices: 0 1 2 3 4 5 6

Enter the Edges in the format 'v1 v2'; -1 for any vertex to exit:
0 1
1 0
0 3
3 0
1 3
3 1
1 2
2 1
3 2
2 3
1 5
5 1
3 4
4 3
2 4
4 2
2 5
5 2
1 6
6 1
4 6
6 4
-1 0

Displaying Adjacency List...
0: 1 3
1: 0 3 2 5 6
2: 1 3 4 5
3: 0 1 2 4
4: 3 2 6
5: 1 2
6: 1 4

Source Vertex: 0
```

SOURCE CODE

Breadth-First Search

```
#include <iostream>
#include <unistd.h>
#include <queue>
#include <vector>
using namespace std;

vector<int>::iterator i;

void printQueue(queue <int> q) {
	queue <int> temp;
	int e;
	while (!q.empty()) {
		e=q.front();
		q.pop();
		temp.push(e);
		cout<<e<<" ";
	}
	cout<<endl;
	while (!temp.empty()) {
		e=temp.front();
		temp.pop();
		q.push(e);
	}
}

void printBFS(vector <int> bfs) {
	for (i=bfs.begin(); i!=bfs.end(); i++)
		cout<<*i<<" ";
		cout<<endl;
}

void bfs (vector < vector<int> > a, int source, vector <bool> visited) {
	queue <int> q;
	visited[source]=true;
	q.push(source);
	int current, count;
```

```
Breadth-First Traversal of Graph:
Current Vertex: 0
Unvisited Neighbours: 1 3
Current Queue: 1 3

BFS: 0

Current Vertex: 1
Unvisited Neighbours: 2 5 6
Current Queue: 3 2 5 6

BFS: 0 1

Current Vertex: 3
Unvisited Neighbours: 4
Current Queue: 2 5 6 4

BFS: 0 1 3

Current Vertex: 2
Unvisited Neighbours: None
Current Queue: 5 6 4

BFS: 0 1 3 2
```

```
	vector <int> b;
	while (!q.empty()) {
		current=q.front();
		q.pop();
		b.push_back(current);
		cout<<"\nCurrent Vertex: "<<current<<endl;
		count=0;
		cout<<"Unvisited Neighbours: ";
		for (i=a[current].begin(); i!=a[current].end(); i++) {
			if (!visited[*i]) {
				cout<<*i<<" ";
				count++;
				q.push(*i);
				visited[*i]=true;
			}
		}
		if (!count)
			cout<<"None";
		cout<<"\nCurrent Queue: ";
		if (q.empty())
			cout<<"Empty"<<endl;
		else
			printQueue(q);
		cout<<"\nBFS: ";
		printBFS(b);
		sleep(2);
	}
}

int main() {
	int n, v1, v2, s;
	cout<<"No. of Vertices: ";
	cin>>n;
	cout<<"\nVertices: ";
	for (int i=0; i<n; i++)
		cout<<i<<" ";
	cout<<endl;
	vector < vector<int> > a(n);
	vector <bool> v(n);
```

```
Current Vertex: 5
Unvisited Neighbours: None
Current Queue: 6 4

BFS: 0 1 3 2 5

Current Vertex: 6
Unvisited Neighbours: None
Current Queue: 4

BFS: 0 1 3 2 5 6

Current Vertex: 4
Unvisited Neighbours: None
Current Queue: Empty

BFS: 0 1 3 2 5 6 4

Breadth-First Search Complete.
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_09$
```

```
	cout<<"\nEnter the Edges in the format 'v1 v2'; -1 for any vertex to
exit:"<<endl;
	while (1) {
		cin>>v1;
		cin>>v2;
		if (v1==-1 || v2==-1)
			break;
		if (v1>=n || v2>=n) {
			cout<<"Vertex doesn't exist."<<endl;
			continue;
		}
		a[v1].push_back(v2);
	}
	cout<<"\nDisplaying Adjacency List..."<<endl;
	for (int i=0; i<n; i++) {
		cout<<i<<": ";
		vector<int>::iterator j;
		for (j=a[i].begin(); j!=a[i].end(); j++)
			cout<<*j<<" ";
		cout<<endl;
	}
	while (1) {
		cout<<"\nSource Vertex: ";
		cin>>s;
		if (s<0 || s>=n) {
			cout<<"Invalid."<<endl;
			continue;
		}
		break;
	}
	cout<<"\nBreadth-First Traversal of Graph: ";
	bfs(a, s, v);
	cout<<"\nBreadth-First Search Complete."<<endl;
	return 0;
}
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_09$ g++ dfs.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_09$ ./a.out
No. of Vertices: 7

Vertices: 0 1 2 3 4 5 6

Enter the Edges in the format 'v1 v2'; -1 for any vertex to exit:
0 1
1 0
0 3
3 0
1 3
3 1
1 2
2 1
3 2
2 3
1 5
5 1
3 4
4 3
2 4
4 2
2 5
5 2
1 6
6 1
4 6
6 4
-1 0

Displaying Adjacency List...
0: 1 3
1: 0 3 2 5 6
2: 1 3 4 5
3: 0 1 2 4
4: 3 2 6
5: 1 2
6: 1 4

Source Vertex: 0
```

Depth-First Search

```
#include <iostream>
#include <stack>
#include <vector>
#include <unistd.h>
using namespace std;

int n;
vector<int>::iterator i;
void printStack(stack <int> s) {
	stack <int> temp;
	int e;
	while (!s.empty()) {
		e=s.top();
		s.pop();
		temp.push(e);
		cout<<e<<" ";
	}
	cout<<endl;
	while (!temp.empty()) {
		e=temp.top();
		temp.pop();
		s.push(e);
	}
}

void printDFS(vector <int> dfs) {
	for (i=dfs.begin(); i!=dfs.end(); i++)
		cout<<*i<<" ";
	cout<<endl;
}

void dfs (vector < vector<int> > a, int source, vector <bool> visited) {
	stack <int> s;
	vector <bool> instack(n);
	vector <int> d;
	s.push(source);
	instack[source]=true;
	int current;
```

```
Depth-First Traversal of Graph

Current Vertex: 0
Unvisited Neighbours: 1 3
Current Stack: 3 1

DFS: 0

Current Vertex: 3
Unvisited Neighbours: 1 2 4
Current Stack: 4 2 1

DFS: 0 3

Current Vertex: 4
Unvisited Neighbours: 2 6
Current Stack: 6 2 1

DFS: 0 3 4

Current Vertex: 6
Unvisited Neighbours: 1
Current Stack: 2 1

DFS: 0 3 4 6
```

```
    while (!s.empty()) {
            current=s.top();
            s.pop();
            d.push_back(current);
            visited[current]=true;
            cout<<"\nCurrent Vertex: "<<current<<endl;
            cout<<"Unvisited Neighbours: ";
            int count=0;
            for (i=a[current].begin(); i!=a[current].end(); i++) {
                    if (!visited[*i]) {
                            cout<<*i<<" ";
                            count++;
                    }
                    if (!visited[*i] && !instack[*i]) {
                            s.push(*i);
                            instack[*i]=true;
                    }
            }
            if (!count)
                    cout<<"None";
            cout<<"\nCurrent Stack: ";
            if (s.empty())
                    cout<<"Empty"<<endl;
            else
                    printStack(s);
            cout<<"\nDFS: ";
            printDFS(d);
            sleep(2);
    }
}

int main() {
    int v1, v2, s;
    cout<<"No. of Vertices: ";
    cin>>n;
    cout<<"\nVertices: ";
    for (int i=0; i<n; i++)
            cout<<i<<" ";
    cout<<endl;
    vector < vector<int> > a(n);
```

```
Current Vertex: 2
Unvisited Neighbours: 1 5
Current Stack: 5 1

DFS: 0 3 4 6 2

Current Vertex: 5
Unvisited Neighbours: 1
Current Stack: 1

DFS: 0 3 4 6 2 5

Current Vertex: 1
Unvisited Neighbours: None
Current Stack: Empty

DFS: 0 3 4 6 2 5 1

Depth-First Search Complete.
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_09$
```

```
    vector <bool> v(n);
    cout<<"\nEnter the Edges in the format 'v1 v2'; -1 for any vertex to
exit:"<<endl;
    while (1) {
        cin>>v1;
        cin>>v2;
        if (v1==-1 || v2==-1)
            break;
        if (v1>=n || v2>=n) {
            cout<<"Vertex doesn't exist."<<endl;
            continue;
        }
        a[v1].push_back(v2);
    }
    cout<<"\nDisplaying Adjacency List..."<<endl;
    for (int i=0; i<n; i++) {
        cout<<i<<": ";
        vector<int>::iterator j;
        for (j=a[i].begin(); j!=a[i].end(); j++)
            cout<<*j<<" ";
        cout<<endl;
    }
    while (1) {
        cout<<"\nSource Vertex: ";
        cin>>s;
        if (s<0 || s>=n) {
            cout<<"Invalid."<<endl;
            continue;
        }
        break;
    }
    cout<<"\nDepth-First Traversal of Graph"<<endl;
    dfs(a, s, v);
    cout<<"\nDepth-First Search Complete."<<endl;
    return 0;
}
```

RESULT

The different graph traversal algorithms – breadth-first search and depth-first search – were successfully implemented.

Exp. No: 11
Date: 16/04/2024

IMPLEMENTATION OF SHORTEST PATH ALGORITHM: DIJKSTRA'S ALGORITHM

AIM

To implement Dijkstra's shortest path algorithm and find the shortest paths to each vertex fron a single source vertex in a given graph.

ALGORITHM

```
edgeWeight (v1, v2, edges, weights)
    for each vertex in edges[v1]
        if vertex is equal to v2
            exit with position of v2
    for each vertex in weights[v1]
        if position has vertex v2 has been reached
            return with weight of edge
dijkstra (edges e, weights w, visited list vl, source s, shortest path list sp)
    sp[s] = 0
    visited[s] = true
    current=s
    while not all vertices are visited
        for each unvisited neighbour of current
            distance = sp[current] + edgeWeight(current,
neighbour, edges, weights)
            if d is less than sp[neighbour]
                sp[neighbour]=d
    min = inf
    pos = -1
    for each index i in sp
        if sp[i] is less than min
            min = sp[i]
            pos = i
    current = pos
    visited[pos] = true
```

SOURCE CODE

```
#include <iostream>
#include <unistd.h>
#include <vector>
using namespace std;

#define inf 1000

void getVertices(int n) {
	cout<<"\nVertices: ";
	for (int i=0; i<n; i++)
		cout<<i<<" ";
	cout<<endl;
}

void displayGraph(vector < vector<int> > edges, vector < vector <int> >
weights, int n) {
	int count;
	cout<<"Vertex\tConnected to\tWeights"<<endl;
	for (int i=0; i<n; i++) {
		count=0;
		cout<<i<<"\t";
		vector<int>::iterator j;
		for (j=edges[i].begin(); j!=edges[i].end(); j++) {
			cout<<*j<<" ";
			count++;
		}
		if (count<=3) cout<<"\t\t";
		else cout<<"\t";
		vector<int>::iterator k;
		for (k=weights[i].begin(); k!=weights[i].end(); k++)
			cout<<*k<<" ";
		cout<<endl;
	}

}
```

```
int edgeWeight(int v1, int v2, vector < vector <int> > edges, vector <vector
<int> > weights) {
	int count=0, pos;
	vector<int>::iterator l;
	for (l=edges[v1].begin(); l!=edges[v1].end(); l++) {
		if (*l==v2) {
			pos=count;
			break;
		}
		count++;
	}
	count=0;
	vector<int>::iterator m;
	for (m=weights[v1].begin(); m!=edges[v1].end(); m++) {
		if (count==pos)
			return *m;
		count++;
	}
	return 0;
}

void displaySP(vector <int> sp, int n) {
	int x;
	for (int i=0; i<n; i++) {
		x=sp[i];
		if (x==inf) cout<<"inf\t";
		else cout<<sp[i]<<"\t";
	}
	cout<<endl;
}

void dijkstra(vector < vector <int> > edges, vector < vector <int> > weights, int
s, vector <bool> visited, vector <int> sp, int n) {
	int current=s, visitedNo=1, d, v, min, pos;
	for (int i=0; i<n; i++) {
		sp[i]=inf;
		visited[i]=false;
	}
	sp[s]=0;
	visited[s]=true;
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_10$ g++ dijkstra.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_10$ ./a.out
No. of Vertices: 9

Vertices: 0 1 2 3 4 5 6 7 8
Enter the edges and edge weights in the format 'v1 v2 weight'; -1 to exit
0 1 4
1 0 4
0 7 8
7 0 8
1 7 11
7 1 11
1 2 8
2 1 8
7 8 7
8 7 7
7 6 1
6 7 1
2 8 2
8 2 2
8 6 6
6 8 6
2 3 7
3 2 7
2 5 4
5 2 4
6 5 2
5 6 2
3 5 14
5 3 14
3 4 9
4 3 9
4 5 10
5 4 10
-1
```

```
	cout<<"\nVertex\t";
	for (int i=0; i<n; i++)
		cout<<i<<"\t";
	cout<<endl;
	while (visitedNo!=n) {
		vector<int>::iterator o;
		for (o=edges[current].begin(); o!=edges[current].end(); ++o) {
			if (!visited[*o]) {
				d=sp[current]+edgeWeight(current, *o, edges, weights);
				if (d<sp[*o])
					sp[*o]=d;
				sleep(1);
			}
		}
		min=inf;
		for (int i=0; i<n; i++)
			if (!visited[i] && sp[i]<min) {
				pos=i;
				min=sp[i];
			}
		cout<<current<<"\t";
		displaySP(sp, n);
		current=pos;
		visited[pos]=true;
		visitedNo++;
	}
}

int main() {
	int n, v1, v2, w, s;
	cout<<"No. of Vertices: ";
	cin>>n;
	getVertices(n);
	vector <int> sp(n);
	vector <bool> visited(n);
	vector < vector<int> > edges(n);
	vector < vector<int> > weights(n);
	cout<<"Enter the edges and edge weights in the format 'v1 v2 weight'; -1
to exit"<<endl;
```

```
Displaying Graph Information...
Vertex  Connected to    Weights
0       1 7             4 8
1       0 7 2           4 11 8
2       1 8 3 5         8 2 7 4
3       2 5 4           7 14 9
4       3 5             9 10
5       2 6 3 4         4 2 14 10
6       7 8 5           1 6 2
7       0 1 8 6         8 11 7 1
8       7 2 6           7 2 6

Source Vertex: 0

Performing Dijkstra's Algorithm...

Vertex  0       1       2       3       4       5       6       7       8
0       0       4       inf     inf     inf     inf     inf     8       inf
1       0       4       12      inf     inf     inf     inf     8       inf
7       0       4       12      inf     inf     inf     9       8       15
6       0       4       12      inf     inf     11      9       8       15
5       0       4       12      25      21      11      9       8       15
2       0       4       12      19      21      11      9       8       14
8       0       4       12      19      21      11      9       8       14
3       0       4       12      19      21      11      9       8       14

Dijkstra's Algorithm Completed.
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_10$
```

```
    while (1) {
        cin>>v1;
        if (v1==-1)
            break;
        cin>>v2;
        cin>>w;
        if (v1<0 || v2<0 || v1>=n || v2>=n) {
            cout<<"Invalid vertex."<<endl;
            continue;
        }
        if (w<0) {
            cout<<"Negative weights are not allowed."<<endl;
            continue;
        }
        edges[v1].push_back(v2);
        weights[v1].push_back(w);
    }
    cout<<"\nDisplaying Graph Information..."<<endl;
    displayGraph(edges, weights, n);
    cout<<"\nSource Vertex: ";
    cin>>s;
    cout<<"\nPerforming Dijkstra's Algorithm..."<<endl;
    dijkstra(edges, weights, s, visited, sp, n);
    sleep(2);
    cout<<"\nDijkstra's Algorithm Completed."<<endl;
    return 0;
}
```

RESULT

Dijkstra’s algorithm was successfully implemented, and the shortest paths from the source vertex to all the other vertices were found successfully.

Exp. No: 12
Date: 23/04/2024

IMPLEMENTATION OF MINIMUM SPANNING TREE: PRIM'S ALGORITHM

AIM

To implement Prim's algorithm and find the minimum cost spanning tree for any given graph.

ALGORITHM

```
edgeWeight (edges, weights, v1, v2)
    for each vertex in edges[v1]
        if vertex is equal to v2
            exit with position of v2
    for each vertex in weights[v1]
        if position has vertex v2 has been reached
            return with weight of edge

prim (edges, weights, no. of vertices n)
    initialise visited list
    initialise edge set – edges chosen by algorithm
    v1, v2 = edge with smallest weight
    while no. of iterations is less than n-1
        visited list [v1] = true
        visited list [v2] = true
        add v1, v2 to edge set
        smallest = inf
        for each current vertex cv that has been visited
            for each neighbouring vertex nv that is unvisited
                weight = edgeWeight (edges, weights, cv, nv)
                if weight is less than smallest
                    smallest = weight
                    v1, v2 = (cv, nv)
```

SOURCE CODE

```
#include <iostream>
#include <unistd.h>
#include <vector>
using namespace std;

#define INF 1000

void getVertices(int n) {
	cout<<"\nVertices: ";
	for (int i=0; i<n; i++)
		cout<<i<<" ";
	cout<<endl;
}

void displayGraph(vector < vector<int> > edges, vector < vector <int> >
weights, int n) {
	int count;
	cout<<"Vertex\tConnected to\tWeights"<<endl;
	for (int i=0; i<n; i++) {
		count=0;
		cout<<i<<"\t";
		vector<int>::iterator j;
		for (j=edges[i].begin(); j!=edges[i].end(); j++) {
			cout<<*j<<" ";
			count++;
		}
		if (count<=3) cout<<"\t\t";
		else cout<<"\t";
		vector<int>::iterator k;
		for (k=weights[i].begin(); k!=weights[i].end(); k++)
			cout<<*k<<" ";
		cout<<endl;
	}

}
```

```
int edgeWeight(vector < vector<int> > edges, vector < vector<int> > weights,
int v1, int v2) {
        int count=0, pos;
        vector<int>::iterator l;
        for (l=edges[v1].begin(); l!=edges[v1].end(); l++) {
                if (*l==v2) {
                    pos=count;
                    break;
            }
            count++;
        }
        count=0;
        vector<int>::iterator m;
        for (m=weights[v1].begin(); m!=edges[v1].end(); m++) {
            if (count==pos)
                return *m;
            count++;
        }
        return 0;
}

void prim (vector < vector<int> > edges, vector < vector<int> > weights, int n,
int vertex1, int vertex2, int weight) {
        vector <bool> visited(n);
        vector < vector<int> > mst(n-1);
        int p=0, w, v1=vertex1, v2=vertex2, smallest=weight, total=0;
        cout<<"\nEdge\tWeight\n"<<endl;
        while (p<n-1) {
                visited[v1]=true;
                visited[v2]=true;
                mst[p].push_back(v1);
                mst[p].push_back(v2);
                mst[p].push_back(smallest);
                total+=smallest;
                smallest=INF;
                for (int i=0; i<n; i++) {
                        vector<int>::iterator o;
                        if (visited[i]) {
                                for (o=edges[i].begin(); o!=edges[i].end(); o++) {
                                        if (!visited[*o]) {
```

```
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_11$ g++ prim.cpp
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_11$ ./a.out
No. of Vertices: 6

Vertices: 0 1 2 3 4 5
Enter the edges and edge weights in the format 'v1 v2 weight'; -1 to exit
0 1 4
1 0 4
0 2 4
2 0 4
1 2 2
2 1 2
2 3 3
3 2 3
2 4 2
4 2 2
2 5 4
5 2 4
3 5 3
5 3 3
4 5 3
5 4 3
-1

Displaying Graph Information...

Vertex  Connected to    Weights
0       1 2             4 4
1       0 2             4 2
2       0 1 3 4 5       4 2 3 2 4
3       2 5             3 3
4       2 5             2 3
5       2 3 4           4 3 3
```

```
                                        w=edgeWeight(edges, weights, i, *o);
                                        if (w<smallest) {
                                                smallest=w;
                                                v1=i;
                                                v2=*o;
                                        }
                                }
                        }
                }
            }
            p++;
        }
        for (int x=0; x<n-1; x++) {
            v1=mst[x][0];
            v2=mst[x][1];
            w=mst[x][2];
            cout<<v1<<"-"<<v2<<"\t"<<w<<endl;
            sleep(1);
        }

}

int main() {
    int n, v1, v2, sv1, sv2, w;
    cout<<"No. of Vertices: ";
    cin>>n;
      getVertices(n);
    vector < vector<int> > edges(n);
    vector < vector<int> > weights(n);
    cout<<"Enter the edges and edge weights in the format 'v1 v2 weight'; -1
to exit"<<endl;
      int smallest=INF;
    while (1) {
        cin>>v1;
            if (v1==-1)
                break;
        cin>>v2;
        cin>>w;
        if (v1<0 || v2<0 || v1>=n || v2>=n) {
                cout<<"Invalid vertex."<<endl;
```

```
Performing Prim's Algorithm...

Edge    Weight

1-2     2
2-4     2
2-3     3
3-5     3
1-0     4

Prim's Algorithm Completed.
warspinix@Yuvaraj-Laptop:~/c++/lab/exp_11$
```

```
                continue;
            }
            edges[v1].push_back(v2);
            weights[v1].push_back(w);
                if (w<smallest) {
                        smallest=w;
                        sv1=v1;
                        sv2=v2;
                }
        }
        cout<<"\nDisplaying Graph Information...\n"<<endl;
        displayGraph(edges, weights, n);
        sleep(1);
        cout<<"\nPerforming Prim's Algorithm..."<<endl;
        prim(edges, weights, n, sv1, sv2, smallest);
        cout<<"\nPrim's Algorithm Completed."<<endl;
        return 0;
}
```

RESULT

Prim’s algorithm was successfully implemented to find the minimum cost spanning tree.